

造物π

基于手势姿态估计与图文双相似度缓存复用的 AR 3D 生成系统

我们团队希望让课堂与展示场景中的现实物体可以被手势选中、快速转化、重复复用。系统通过手势姿态估计完成自然交互，通过图文双相似度缓存复用减少重复 3D 生成等待，最终提升 AR 3D 展示体验。

一、团队视角与项目出发点

我们把本项目定位为一个面向课堂、实验室和产品展示场景的 AI + AR 工具。实际教学中，许多实物、教具和实验器材没有现成数字化素材；如果每次依赖人工建模，门槛较高；如果完全依赖 AI 3D 生成，等待时间又较长。因此，我们希望做一个轻量的交互系统，让用户通过手势选中现实物体，系统自动完成 ROI 截取、语义识别、3D 模型加载和缓存复用。

项目核心不是单纯追求更大的生成模型，而是把“已生成过的高成本 3D 结果”沉淀为可复用资产。对于相同或相似物体，系统优先复用已有 GLB 模型，减少重复调用生成模型，从而实现更适合本地设备和课堂场景的轻量化算力优化。

二、AI 多模态能力与助学价值

本项目的 AI 能力体现在一条多模态链路中：系统同时处理摄像头画面、手部姿态、ROI 图像、物体语义文本和 3D 模型资产，并根据图文相似度做缓存复用决策。对使用者而言，AI 能力被隐藏在自然交互背后：用户只需要通过手势框选目标，系统会自动理解用户意图、定位目标区域，并判断是否可以复用已有 3D 模型。

模态/能力	在项目中的作用	对应 AI 能力
视频流	采集真实课堂或桌面场景	计算机视觉输入
手部关键点	识别用户框选、拖动、缩放等动作	MediaPipe 手势姿态估计
ROI 图像	截取被用户选中的目标物体	图像理解与视觉特征提取
语义文本	形成 keyword 与 query_text	物体语义理解
图文相似度	判断当前物体是否可复用已有模型	多模态匹配与缓存决策
GLB 模型	在前端展示可旋转、可观察的 3D 资产	AR 展示与数字素材复用

三、技术链路图

系统采用“手势触发—ROI 框选—语义理解—缓存判断—模型展示”的端到端链路。图文双相似度策略位于模型展示之前，用于判断当前目标是否可以复用已有 GLB 缓存，从而把生成过的模型沉淀为可持续复用的数字资产。

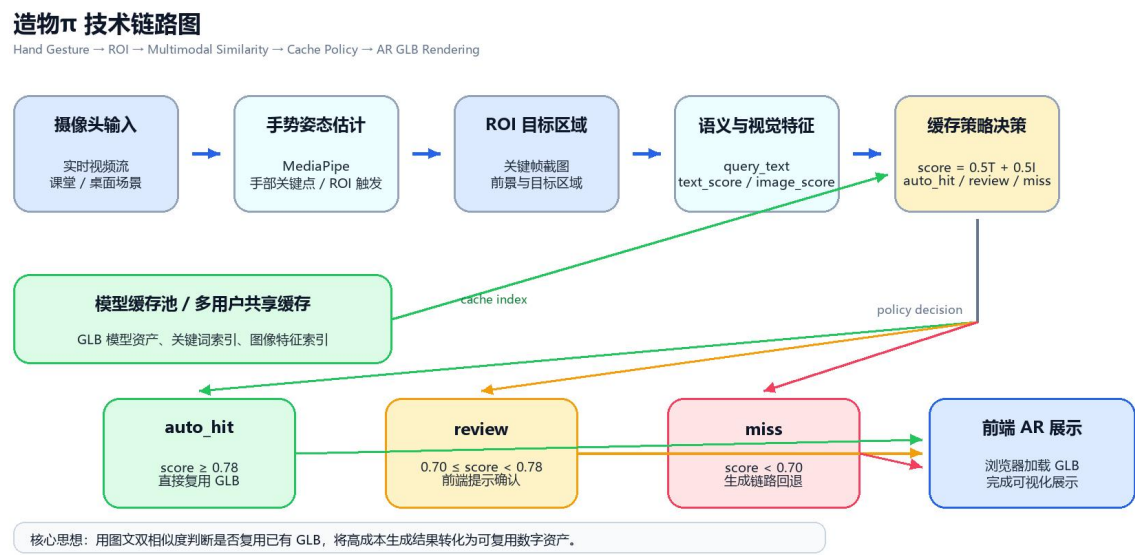


图 1 系统技术链路与缓存复用策略框图



图 2 系统页面与高质量 GLB 展示效果截图

四、系统功能与实现

- 自然交互：通过手势姿态估计完成目标选择，降低操作门槛。
- ROI 截图：从摄像头画面中截取目标区域，为识别、生成和缓存检索提供输入。
- 3D 展示：前端加载 GLB 模型，支持课堂、实验和产品展示场景。
- 缓存复用：把已生成或已准备的模型沉淀为可重复使用资产。
- 轻量优化：减少重复调用 3D 生成模型，降低本地算力压力和等待时间。

比赛演示优先使用本地已有 GLB 和高质量预置展示资产，保证现场画面稳定；同时通过 baseline 耗时对比说明缓存复用带来的速度收益。这样的设计既能展示 AI 交互链路，也能说明缓存复用确实减少了等待。

五、缓存复用策略与实验数据

缓存复用策略使用文本相似度和图像相似度共同判断当前物体是否可以复用已有模型。当前规则融合公式为：
 $score = 0.5 \times text_score + 0.5 \times image_score$ 。根据分数，系统分为 auto_hit、review 和 miss 三个分支。

分支	判断条件	系统行为	当前设计
auto_hit	$score \geq 0.78$	直接复用缓存 GLB	自动复用，跳过生成
review	$0.70 \leq score < 0.78$	提示存在相似缓存	前端展示确认提示
miss	$score < 0.70$	不复用缓存	回退原始生成流程

在 v3_real_70 样本上，基础策略结果为 $accuracy = 0.8$, $precision = 1.0$, $recall = 0.6111$, $false_hit_rate = 0.0$, $false_miss_rate = 0.3889$ 。双阈值分析显示， $weak_threshold = 0.70$ 、 $strong_threshold = 0.78$ 时，可以将 10 个边界样本纳入 review 区，同时保持 $auto_false_hit_rate = 0$ 。

方法	recall	false_hit_rate	near_positive_false_miss_rate	结论
text-only	0.6111	0.0	0.3333	可解释性较强，但依赖文本标签质量
image-only	0.3333	0.0294	0.75	召回偏低，且存在误命中风险
rule-fusion 0.5 / 0.5	0.6111	0.0	0.3333	整体稳定，适合作为规则基线
dual-threshold fusion	0.6111	0.0	0.3333	可将边界样本拆入 review 区

六、速度收益与轻量化算力优化

这部分结果是项目的核心价值之一：我们并不是让生成模型本身变快，而是通过智能缓存判断减少不必要的生成调用。对于本地设备和课堂场景来说，这种做法更轻量，也更容易落地。

项目	数值	说明
baseline_triposr_elapsed_ms	51236	TripoSR baseline 单次生成耗时
auto_hit_total_elapsed_ms	3403.165	缓存复用路径耗时
saved_latency_ms	47832.835	单样本节省约 47.8 秒
生成调用状态	已跳过	auto_hit 路径直接复用缓存 GLB

多用户仿真指标	数值
total_samples	70
generation_ms	51236
network_delay_ms	20000
saved_latency_seconds	663.902
speedup_ratio	1.2272

多用户仿真说明了系统在协作场景中的潜力。即使远端模型传输延迟设置为 20 秒，只要能复用其他用户已经生成过的模型，总体上仍然比每次都重新生成更快。

七、链路构建与实验推进过程

本项目的推进过程不是一次性完成全部功能，而是按照“先打通系统链路，再验证缓存策略，最后回到产品演示”的节奏逐步构建。这样做的好处是，每一阶段都有明确目标：先保证能交互、能截取、能展示，再证明缓存复用确实有效，最后把技术结果整理成可运行 Demo 和比赛材料。

阶段	主要工作	形成结果
基础交互链路	搭建摄像头页面，接入 MediaPipe 手势姿态估计，完成 ROI 框选与目标触发	系统具备“用手势选择现实物体”的交互入口
模型展示链路	整理 GLB 模型加载方式，打通前端模型展示和缓存模型访问路径	浏览器页面可以展示可旋转、可观察的 3D 模型
缓存判断链路	从关键词缓存扩展到图文双相似度判断，记录 text_score、image_score 和 fused_score	系统具备判断相似目标是否可复用的能力
双阈值策略	构建 auto_hit、review、miss 三支策略，并通过离线实验选择阈值	高相似目标自动复用，边界目标进入提示区，低相似目标回退生成
工程接入验证	将策略以开关形式轻量接入 plus.py，并完成三支小样本验证	策略逻辑可以在真实业务路径中运行

展示与交付	整理预置 GLB、高质量展示资产、演示视频和说明文档	形成可提交的比赛作品包
-------	----------------------------	-------------

实验部分也按照由小到大的方式推进。最初使用 v2_hard 小样本集验证 hard_negative 对误复用风险的影响；随后扩展到 v3_real，逐步补充 positive、near_positive、hard_negative 和 negative 样本；最后形成 v3_real_70 分支，用 70 条样本复核相似度策略、双阈值区间、错误样本和速度收益。这个过程保证了策略不是只在单个样本上有效，而是在不同类型样本上都经过了离线验证。

实验环节	目的	关键结论
v2_hard 小样本实验	验证 hard_negative 场景下的误复用风险	单阈值可以保证安全，但 near_positive 召回偏保守
融合权重消融	比较 text-only、image-only 和图文融合权重	规则融合比单一模态更适合作为可解释 baseline
v3_real 扩样本	扩大真实样本数量，验证策略稳定性	扩样本后 recall 提升，false_hit_rate 保持为 0
边界样本重扫	分析 review 区间为何为空，并重新扫描 weak / strong 阈值	0.70 / 0.78 可以让边界样本进入 review 区
人工复核 review 样本	确认 review 区是否混入负样本	10 个 review 样本均适合作为用户确认区候选
三支运行验证	验证 auto_hit、review、miss 在工程路径中的行为	auto_hit 可复用，review 保守提示，miss 安全回退
速度收益验证	确认 auto_hit 是否真的跳过生成调用	单样本节省约 47.8 秒，证明缓存复用带来实际速度收益

在工程实现上，项目始终保持“主链路可回退”的原则。缓存策略通过配置文件读取，ENABLE_POLICY_CACHE 控制是否启用；当策略关闭时，系统保持原流程；当策略开启时，只在融合相似度计算后进入 policy decision；review 分支先展示提示并保持保守处理。这样的设计让项目既能展示研究创新，也不会破坏比赛 Demo 的稳定性。

策略分支	实验验证	工程行为	比赛展示方式
auto_hit	policy_score = 1.0, auto_reuse = True	直接加载缓存 GLB	展示快速复用和速度收益
review	policy_score = 0.75, review 样本通过人工复核	前端显示相似缓存提示	展示边界样本可解释提示
miss	policy_score = 0.0, auto_reuse = False	进入生成回退链路	展示低相似目标不会误复用

从系统架构角度看，造物π可以分为四层：交互输入层、智能判断层、模型资产层和展示反馈层。交互输入层负责把现实世界中的物体转化为 ROI 图像；智能判断层负责把 ROI 图像和语义文本转化为可解释的相似度分数；模型资产层负责管理缓存 GLB、预置 GLB 和共享模型；展示反馈层负责把决策结果以模型加载、review 提示或生成回退的方式反馈给用户。四层之间保持相对解耦，使系统既能运行比赛 Demo，也便于后续替换更高质量的 3D 生成后端。

系统层级	输入	核心处理	输出
交互输入层	摄像头视频流、用户手势	手部关键点识别、ROI 框选、关键帧截取	ROI 图片与交互状态
智能判断层	ROI 图片、query_text、缓存索引	文本相似度、图像相似度、policy_score 计算	auto_hit / review / miss 决策
模型资产层	缓存 GLB、预置 GLB、模型路径	模型索引、路径映射、多用户缓存仿真	可加载的 GLB 资产
展示反馈层	策略决策与 GLB 路径	前端模型加载、提示卡片、状态日志	浏览器 AR 展示效果

比赛演示链路围绕“完整可看、重点突出、风险可控”设计。首先展示摄像头页面和手势识别，让评委看到系统不是静态网页；随后展示 ROI 框选和物体识别，说明系统能从现实画面中提取目标；再展示 GLB 模型加载，呈现 AR 3D 可视化效果；最后展示缓存复用和速度收益，让评委理解项目创新点不是单纯展示模型，而是通过图文双相似度减少重复生成等待。

演示环节	展示内容	对应创新点
手势交互	摄像头画面、手部关键点、ROI 框选	自然交互入口
目标理解	ROI 图像、物体语义、query_text	多模态识别链路
模型展示	浏览器加载 GLB，形成可视化展示	AR 3D 展示能力
缓存复用	auto_hit 直接加载已有模型	图文双相似度提速
边界提示	review 提示卡片	可解释、安全的边界处理
协同仿真	多用户缓存节省时间结果	共享缓存的扩展价值

实验数据和产品结论之间是一一对应的。false_hit_rate 用于说明自动复用的安全性；recall 和 false_miss_rate 用于说明系统仍有持续优化空间；auto_hit 速度验证用于说明缓存复用确实可以减少等待；多用户仿真用于说明当模型缓存被多人共享时，系统收益会进一步扩大。因此，实验不是孤立指标，而是直接支撑产品可行性的证据链。

实验数据	证明的问题	对应产品结论
false_hit_rate = 0.0	自动复用路径没有误复用样本	缓存复用具备安全基础
recall = 0.6111	系统可以召回一部分相似目标	规则策略已经具备初步实用性
auto_hit 节省约 47.8 秒	高相似样本可跳过重复生成	缓存复用能带来真实速度收益

review_count = 10	边界样本可以被单独识别	系统可以提供更细粒度的人机确认入口
多用户仿真节省约 663.902 秒	共享缓存比单用户重复生成更有价值	适合扩展为团队级素材库

八、系统运行与交付状态

目前系统已经完成缓存策略轻量接入、auto_hit / review / miss 三支业务路径验证、Review UI 提示验证，以及 auto_hit 单样本速度收益验证。本地也已盘点到可用于比赛展示的 GLB 文件，并准备了高质量预置资产用于稳定展示。

资产类别	建议文件名	作用
眼镜 / 太阳镜	glasses.glb / sunglasses_hq.glb	auto_hit 缓存复用主示例
键盘	keyboard.glb	第二类物体展示
杯子 / 水瓶	water_bottle_hq.glb	普通物体展示
音箱 / 盒子	boombox_hq.glb	补充展示资产
头盔 / 相机	damaged_helmet_hq.glb / antique_camera_hq.glb	高质量多类别展示

九、产品方案方向

造物π的产品定位是“面向教学与展示场景的轻量化 AR 3D 内容生成与复用平台”。它解决的不是单次生成一个模型的问题，而是把现实物体数字化、模型资产复用和多人共享连接成一个完整产品闭环：用户通过手势选中目标，系统完成识别、检索、复用和展示，让现实物体快速进入可交互的 3D 展示空间。

在传统教学展示中，教师往往需要提前准备图片、视频或静态模型；在实验室和创客展示中，团队也需要花时间寻找或制作 3D 素材。造物π把这个过程前置为一个自然交互入口：用户面对摄像头，通过手势选择物体，系统把“现实中的物体”转换为“可展示、可复用、可共享的数字资产”。因此它既是一个 AR 演示工具，也是一个轻量的 3D 资产沉淀工具。

产品要素	方案设计	价值体现
目标用户	教师、学生、实验助教、科普活动组织者、课程项目团队	覆盖课堂讲解、实验展示、作品路演和创客活动
核心问题	3D 素材准备成本高、现场生成等待长、相似物体重复生成浪费算力	把生成成本转化为可沉淀的模型资产
产品形态	浏览器 AR 页面 + 手势交互 + 模型缓存库 + 策略判断服务	安装和使用门槛低，适合比赛现场和课堂环境

核心体验	手势框选目标，系统识别物体并自动选择加载缓存、提示确认或进入生成链路	从“拍摄物体”自然过渡到“展示 3D 资产”
差异化能力	图文双相似度缓存复用与多用户共享缓存	在保证误复用风险可控的前提下降低等待时间

典型使用场景	用户流程	适配价值
课堂讲解	教师将教具或实验器材放到摄像头前，手势框选后展示对应 3D 模型	把抽象讲解变成可视化、可旋转、可重复展示的内容
实验演示	学生或助教选择设备、零件或样品，系统加载已有模型或缓存模型	让实验对象更容易被观察和比较
作品路演	团队展示实物原型，通过 AR 页面展示对应数字模型	增强展示的科技感和交互感
科普活动	观众通过手势选中现场物体，系统快速呈现 3D 展示效果	提升参与感，形成可互动的科普体验
多人协作	同一教室或团队成员共用模型缓存库	减少重复准备素材和重复生成等待

产品体验闭环可以概括为四步：看见、选中、理解、复用。用户先在摄像头画面中看到真实物体，再通过手势框选目标；系统对 ROI 图像进行语义理解，同时检索已有 GLB 模型缓存；最后由图文双相似度策略决定展示方式。高相似目标直接加载已有模型，边界目标展示确认提示，低相似目标进入生成链路。这个闭环让每一次生成结果都可以沉淀为后续可复用的数字资产。

功能模块	用户看到的能力	背后的技术支撑
手势交互	用手势选择现实物体，完成 ROI 框选	MediaPipe 手势姿态估计、OpenCV 图像处理
智能识别	系统理解目标类别和语义描述	ROI 图像、query_text、关键词匹配
缓存复用	相似物体快速显示已有 3D 模型	text_score、image_score、policy_score、双阈值策略
边界提示	发现相似模型时给出确认提示	review 分支与前端 cache_review 提示卡片
AR 展示	浏览器中展示可观察的 GLB 模型	Three.js / GLB 加载与模型缓存资产
多用户共享	团队成员可复用同一批模型资产	共享缓存池、网络延迟仿真、复用收益评估

用户流程	界面表现	系统响应
打开页面	浏览器显示摄像头画面和模型展示区域	初始化手势识别、缓存策略和前端渲染模块
手势选择	用户用手势框定目标区域	捕获 ROI 图像并生成识别输入

智能判断	页面展示识别结果或提示状态	计算文本相似度、图像相似度和 policy_score
快速展示	高相似目标直接显示已有 GLB	auto_hit 分支跳过重复生成
边界提示	页面显示“发现相似缓存模型”提示卡片	review 分支提供可解释的确认入口
资产沉淀	模型进入缓存库，可被后续样本复用	形成可持续积累的教学与展示素材

面向落地使用，产品可以形成三个层级的交付形态。第一层是单机演示版，完成摄像头、手势、ROI、GLB 展示和缓存复用；第二层是课堂协作版，增加班级或实验室共享模型库，让多个用户复用同一批模型资产；第三层是内容平台版，进一步接入更高质量的 3D 生成后端和素材管理能力，形成可持续积累的教学与展示资源库。

落地阶段	主要能力	交付成果
单机演示版	手势选择、ROI 截图、缓存 GLB 加载、auto_hit / review / miss 展示	可运行网页 Demo、演示视频、说明文档
课堂协作版	多用户共享缓存、模型资产复用、课堂素材库管理	班级或实验室共享模型库
内容平台版	更高质量生成后端、素材检索、资产沉淀、复用统计	面向教学与展示的 AR 3D 素材平台

产品优势	具体体现
低门槛交互	使用摄像头和手势完成目标选择，减少复杂软件操作。
轻量化算力优化	通过缓存复用减少重复 3D 生成调用，把等待时间转化为资产复用收益。
可解释策略	auto_hit、review、miss 三支清晰，适合向用户和评委说明系统行为。
可扩展资产库	GLB 模型可以持续积累，并扩展到班级、实验室或团队共享。
适合比赛展示	浏览器即可演示完整链路，便于录屏、现场展示和文档说明。

产品方案的核心不是把所有能力堆在一个页面里，而是形成清晰的使用路径：用户在现实场景中看到物体，通过手势完成选择，系统理解目标后优先调用已有模型资产，并在网页端完成 3D 展示。对于评审和实际用户来说，这条链路的价值在于“可运行、可解释、可复用”。

用户痛点	传统方式	造物π方案	改进效果
准备 3D 素材麻烦	手动找模型、建模或等待生成	将 GLB 缓存沉淀为可复用资产	减少重复准备成本
课堂互动弱	学生只能看图片或视频	通过手势选中真实物体并展示	提升参与感和直观性

		3D 模型	
生成等待长	每次相似物体都重新生成	auto_hit 直接复用缓存模型	降低等待时间
边界样本难处理	系统要么误复用，要么全部重新生成	review 区提示用户确认	兼顾安全性和召回空间
团队素材分散	每个人单独准备模型	多用户共享缓存池	让模型资产持续积累

作为参赛作品，造物π不仅展示了一个功能点，而是展示了从交互入口、AI 判断、模型资产到展示交付的完整产品链路。评委可以通过固定 Demo 链接看到前端交互，通过说明文档理解技术策略，通过实验数据看到速度收益，通过源码与复现说明了解工程结构。

产品指标	当前结果	说明
交互完整度	摄像头、手势、ROI、GLB 展示已形成闭环	支持完整演示视频和页面体验
缓存安全性	v3_real_70 false_hit_rate = 0.0	自动复用路径保持较低误复用风险
召回能力	v3_real_70 recall = 0.6111	能够覆盖一部分 positive 与 near_positive 样本
速度收益	auto_hit 单样本节省约 47.8 秒	高相似样本可直接复用缓存模型
协同潜力	多用户仿真节省约 663.902 秒	共享缓存有进一步扩展价值

比赛交付物	内容	作用
可运行 Demo	固定网页入口、摄像头页面、手势交互、GLB 展示、缓存策略提示	让评委直观看到系统可用性
演示视频	手势识别、ROI 框选、模型展示、缓存复用和速度收益讲解	补充展示完整流程
说明文档	项目背景、技术链路、产品方案、实验数据和源码复现说明	帮助评委快速理解创新点
源码与复现说明	核心模块、运行链路、实验目录和固定 Demo 地址	支撑作品可解释和可复核

路线	近期目标	中期目标	长期目标
交互体验	稳定摄像头、手势框选和提示卡片	加入 review 选择按钮和用户反馈记录	形成面向课堂的可配置交互模板
模型资产	整理比赛展示 GLB 与缓存 GLB	建设班级 / 实验室模型素材库	形成可搜索、可管理、可复用的 3D 素材平台
智能策略	使用规则融合与双阈值策略	扩大样本后引入轻量分类器	根据真实日志持续优化复用判断
生成后端	保留 baseline 与预置展示	适配更高质量 image-to-	实现多后端可切换生成服

	资产	3D 后端	务
--	----	-------	---

从更长远的产品愿景看，造物π可以发展为“校级 3D 教学资产中台”。每一次课堂演示、实验展示或科普活动产生的模型，都可以进入学校或团队的共享素材库；后续遇到相似物体时，系统优先复用已有资产，并通过图文相似度保证复用过程可解释、可追踪、可控制。随着素材库不断扩大，系统会从单次 Demo 工具逐步变成可持续增长的教学数字资产平台。

未来产品能力	设想形态	应用前景
智能课堂素材库	按课程、实验、物体类别管理 GLB 模型	教师可以快速调用已有 3D 教具，形成可复用课堂资源
多人协作缓存	同一班级、实验室或项目组共享模型缓存	团队成员减少重复生成和重复整理素材
本地 + 云端混合部署	常用模型本地缓存，高质量生成任务云端处理	兼顾响应速度、展示质量和设备成本
可解释 AI 判断	保留 text_score、image_score、policy_score 与决策日志	便于教学、演示和评审场景解释系统行为
开放式资产接入	支持外部 GLB、课程模型库和更高质量生成后端	把系统扩展为可连接多种 3D 内容来源的平台

这一产品方向也具有较强的教育普惠价值。传统 3D 内容制作需要建模经验和较高设备门槛，而造物π把复杂流程压缩到“手势选择 + 智能复用 + 前端展示”。对于普通课堂，它可以作为低成本 AR 教具；对于课程项目，它可以作为 AI 多模态实践平台；对于创客展示，它可以作为互动式 3D 展示工具。

推广阶段	目标场景	核心动作	预期效果
校内试点	课程展示、实验课堂、创新实践课	沉淀 20 ~ 50 个常用教学 GLB 资产	形成基础素材库和稳定演示流程
团队共创	学生项目组、实验室、创客社团	多人共享缓存，记录复用日志	验证多用户缓存复用价值
平台化扩展	跨课程、跨班级、跨实验室	接入账号、素材分类、使用统计和云端生成后端	形成可持续增长的 AR 教学资产平台

十、展示资产与界面效果

为了让比赛展示更加直观，系统准备了多类 GLB 展示资产。它们用于呈现前端 AR 加载效果、缓存复用效果和多类别扩展能力。下列图片展示了当前可用于演示的视频素材和模型预览，配合固定 Demo 链接可以形成完整的作品展示材料。



图 3 比赛 Demo 页面整体效果

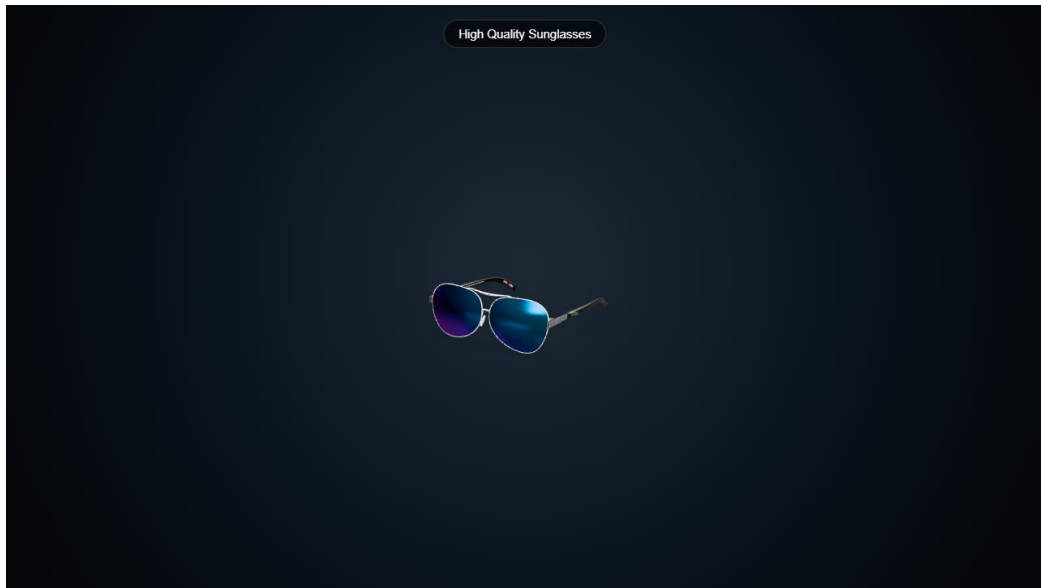


图 4 太阳镜高质量展示资产

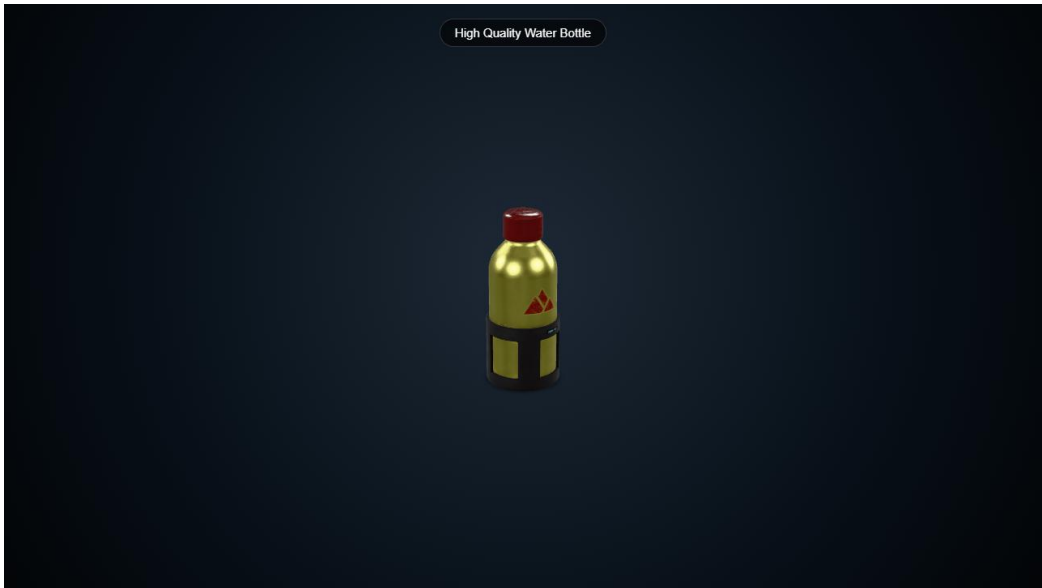


图 5 水瓶展示资产

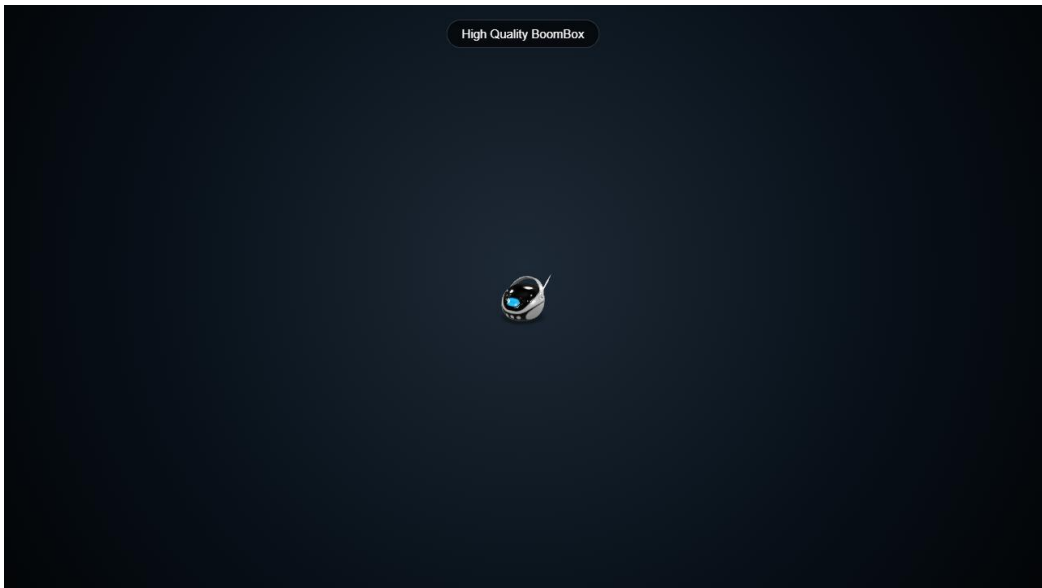


图 6 音箱展示资产



图 7 相机展示资产

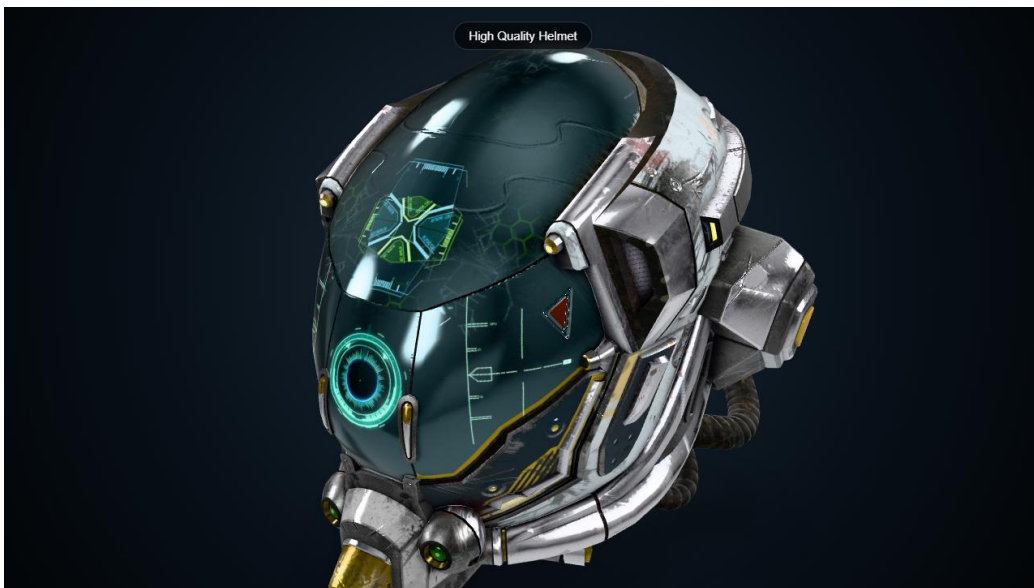


图 8 头盔展示资产

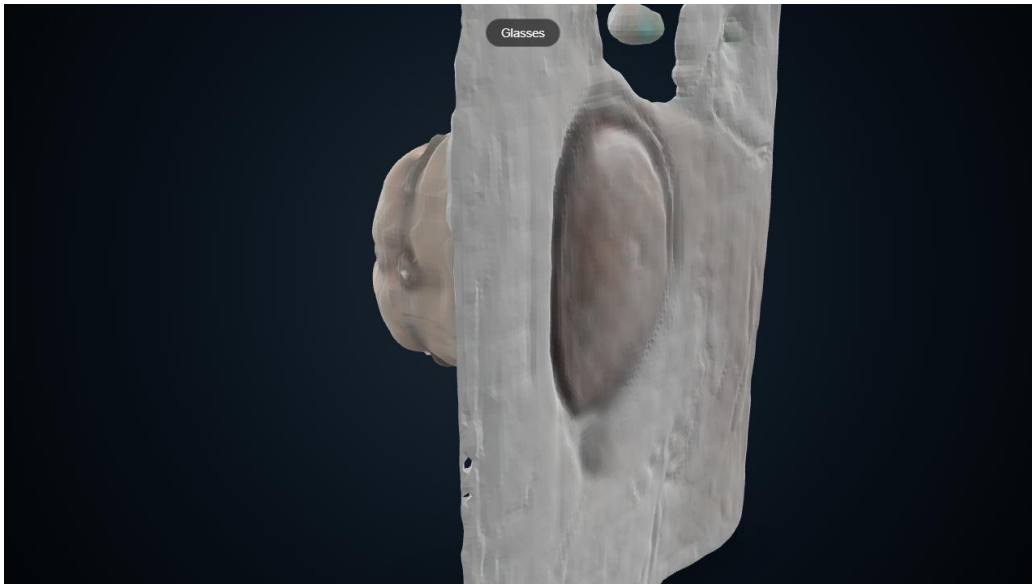


图 9 本地缓存眼镜模型预览

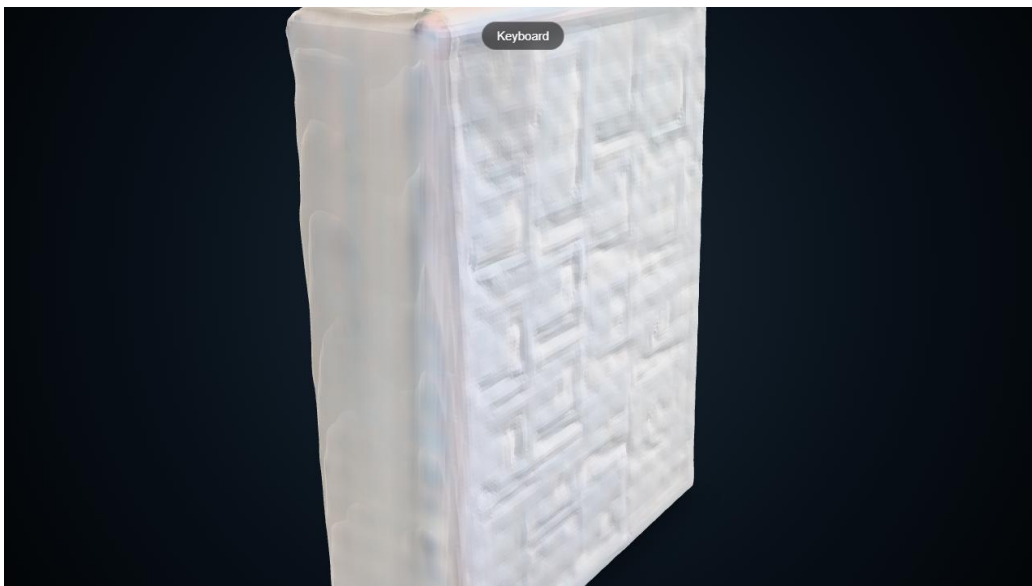


图 10 本地缓存键盘模型预览

十一、产品方案深化与落地设计

在比赛交付层面，造物π不只是一个算法实验或单个网页 Demo，而是一个面向教学、科普和创客展示的完整产品方案。它把摄像头输入、手势交互、ROI 框选、物体语义理解、3D 模型资产加载和缓存复用策略组织成一条端到端链路。用户看到的是一个自然的 AR 交互过程，系统背后完成的是多模态判断、模型资产检索和轻量化算力优化。

产品方案的核心价值可以概括为三点。第一，降低 3D 内容进入课堂和展示场景的门槛，让用户无需掌握建模软件也能看到可旋转、可展示的 3D 资产。第二，把一次生成或整理得到的模型沉淀为后续可复用资产，

使系统越用越快、越用越丰富。第三，用图文双相似度和双阈值策略控制复用风险，在追求速度的同时保持展示准确性。

产品层	设计内容	面向用户的价值
交互入口	摄像头画面、手势识别、ROI 框选	用自然动作选择现实物体，降低使用门槛
智能判断	文本相似度、图像相似度、双阈值策略	决定自动复用、提示确认或重新生成
模型资产	预置 GLB、缓存 GLB、后续可替换生成后端	保证比赛展示稳定，也保留技术扩展空间
前端呈现	浏览器加载 GLB、Review 提示卡片、演示页面	让评委直观看到 AR 3D 展示效果
数据闭环	实验指标、运行日志、多用户仿真	证明系统不是静态展示，而是可持续优化

11.1 用户角色与使用场景

造物π面向的用户并不局限于单一角色。教师可以把它作为课堂演示工具，学生可以把它作为课程项目或创新实践工具，科普活动组织者可以把它作为互动展示入口，项目团队也可以把它作为路演时的可视化增强工具。不同角色关注点不同，但都围绕同一件事：把现实物体更快地转换为可展示、可复用、可讲解的 3D 内容。

用户角色	典型任务	系统提供的帮助	可展示效果
教师	讲解实体教具、实验器材或结构对象	通过手势选中物体并展示 3D 模型	课堂内容更直观，学生更容易理解
学生	完成课程设计、创新实验或作品展示	快速加载已有模型资产并形成展示页面	减少准备素材的时间
实验助教	展示实验装置、零件和操作对象	复用已缓存模型，减少重复等待生成	实验讲解更稳定
科普讲解员	让观众参与互动选择现实物体	用浏览器页面即时展示 3D 效果	增强现场互动感
项目团队	比赛路演、产品演示、技术验证	组合手势交互、模型展示和缓存加速	形成完整可讲述的技术闭环

11.2 产品功能结构

从产品结构看，系统可以拆成六个相互协作的功能区：实时输入区、目标选择区、智能判断区、模型资产区、展示反馈区和实验验证区。这样的结构有利于比赛演示，也方便后续工程迭代。即使后续更换 3D 生成后端，或者把 Review 区升级为真正的用户确认交互，整体架构也可以平滑延续。

功能区	当前能力	后续扩展
-----	------	------

实时输入区	摄像头页面和手势识别入口	可扩展为平板、手机或外接摄像头场景
目标选择区	ROI 框选和截图	增加多目标选择、目标锁定和撤销操作
智能判断区	图文双相似度、auto_hit / review / miss	接入更多类别词表、学习型评分或人工反馈
模型资产区	本地 GLB、缓存 GLB、比赛预置模型	建设课程素材库和团队共享模型库
展示反馈区	浏览器加载 GLB、Review 提示卡片	支持复用确认按钮、模型收藏和展示模板
实验验证区	离线指标、速度验证、多用户仿真	形成长期日志统计和产品运营指标

11.3 端到端体验流程

完整体验可以被设计为一条清晰的用户旅程：用户先打开网页并授权摄像头；随后在真实环境中用手势选中物体；系统截取 ROI 并生成语义描述；缓存策略判断是否存在相似模型；如果是高相似目标，直接加载 GLB；如果是边界目标，显示 Review 提示；如果是低相似目标，则进入生成或兜底展示流程。比赛演示中，为了稳定性，生成部分可以用讲解和已有 GLB 表现，但缓存复用和前端展示链路保持真实可运行。

步骤	用户动作	系统动作	演示重点
1	打开固定 Demo 地址	初始化页面、策略配置和模型展示区域	证明系统可访问、可运行
2	面对摄像头展示手势	识别手部关键点并准备 ROI 交互	体现自然交互入口
3	框选目标物体	截取 ROI，进入识别和缓存判断	体现现实物体到数字对象的转换
4	等待判断结果	计算 text_score、image_score 和 policy_score	体现图文双相似度策略
5	查看模型展示	加载缓存 GLB 或预置 GLB	体现 AR 3D 展示效果
6	查看提示或日志	展示 auto_hit、review 或 miss 状态	体现速度收益和安全回退

11.4 产品体验细节

比赛 Demo 的体验设计需要尽量减少现场不确定性。页面无需把技术细节堆满屏幕，而是像一个正常产品一样，把关键状态用简洁提示呈现给用户。例如 auto_hit 时强调“已复用缓存模型”，Review 时提示“发现相似缓存模型”，miss 时保持原流程。技术指标可以放在说明文档和演示讲解里，前端页面只保留必要反馈。

界面元素	展示内容	设计原则
摄像头画面	实时视频、手势区域、ROI 框选	保持画面清晰，突出交互对象
模型展示区	GLB 模型、旋转视角、加载状态	展示模型为主，不让文本遮挡主体

缓存状态提示	auto_hit / review / miss 对应提示	少量文字解释即可，保持演示画面清爽
Review 提示卡片	相似模型关键词、分数、阈值	第一阶段只提示，不执行真实复用
速度收益说明	51.236 秒 baseline 与 3.403 秒 auto_hit	放在讲解和文档中，页面可轻量显示

11.5 数据闭环与资产增长机制

本项目的一个重要思路是把模型生成结果视为可积累资产，而不是一次性输出。每当系统识别出一个物体并加载或生成模型，就可以把对应的图像特征、文本语义、GLB 路径、复用决策和运行耗时记录下来。随着使用次数增加，系统中的模型资产和缓存判断样本都会增长，后续既可以继续优化阈值，也可以为轻量分类器提供训练基础。

数据来源	记录内容	可产生的后续价值
ROI 图像	目标截图、样本类别、采集来源	扩充真实场景样本，改进相似度评估
语义文本	category、query_text、best_keyword	优化关键词归一化和文本相似度
缓存判断	text_score、image_score、policy_score、decision	分析误命中、漏命中和 Review 区间
模型资产	GLB 路径、来源、展示可用性	形成可复用课程和展示素材库
运行耗时	缓存加载耗时、生成耗时、节省时间	量化速度收益和多用户共享价值

11.6 面向比赛展示的产品包装

比赛展示不只是把功能跑出来，还要让评委在有限时间内理解“为什么这个系统值得做”。因此我们把演示内容组织为三层：第一层展示能用，包含摄像头、手势、ROI 和 GLB 加载；第二层展示有用，强调缓存复用减少等待；第三层展示可扩展，说明多用户缓存和未来素材库方向。这样，评委既能看到产品效果，也能看到技术创新和后续落地空间。

展示层次	要传达的信息	对应素材
能用	系统可以打开页面并完成手势到模型展示的基本链路	演示视频、页面截图、预置 GLB
有用	缓存复用可以减少重复生成等待	auto_hit 速度验证、策略日志、对比数据
可信	误复用风险通过双阈值和 Review 区控制	v3_real_70 实验、三分支验证、Review UI 验证
可扩展	模型资产可沉淀，多用户可共享	多用户仿真结果、产品方案、未来规划

11.7 产品化实施路线

如果后续从比赛作品继续推进到实际可用产品，可以按三个阶段实施。第一阶段保持当前浏览器 Demo 形态，重点完善展示体验和 Review 提示；第二阶段建设资产管理能力，让常用模型可以被分类、检索和复用；第三阶段再接入更高质量 3D 生成后端和更多用户协同能力。这样推进的好处是每一步都有可交付结果，不会因为追求一次性大模型能力而影响主线稳定。

阶段	建设重点	关键交付	评价方式
阶段一：比赛演示	固定 Demo、预置 GLB、缓存策略、Review 提示	可运行网页、演示视频、说明文档	现场可演示、视频可说明
阶段二：小范围试用	资产目录、日志记录、Review 选择、更多真实样本	班级或实验室模型库	用户是否愿意复用模型资产
阶段三：平台化	多用户共享缓存、云端生成、高质量模型后端	课程级 AR 3D 素材平台	复用率、节省时间、资产增长数量

11.8 运行模式设计

为了适配不同场景，系统可以形成四种运行模式。单机模式适合比赛与个人演示；课堂模式适合教师在局域网内共享模型资产；云端增强模式适合把高质量生成任务交给更强算力；素材库模式适合长期沉淀课程与活动资源。这些模式不是互相排斥的，而是可以随着应用深度逐步叠加。

运行模式	适用场景	核心特点	部署要求
单机演示模式	比赛现场、个人汇报、课堂试讲	本地浏览器运行，使用已有 GLB 和缓存策略展示	一台电脑、摄像头、浏览器
课堂共享模式	同一班级或实验室多人使用	共享模型缓存，减少重复生成和重复准备素材	局域网或统一服务器
云端增强模式	需要更高质量模型或更大算力	本地负责交互，云端负责高质量生成	稳定网络和云端 GPU
素材库模式	长期课程建设和科普活动	按类别管理 3D 资产，支持检索与复用	资产管理页面和权限机制

11.9 未来展望

未来，造物π可以继续向“智能 3D 教学资产平台”发展。当前系统已经证明了手势交互、模型展示、缓存复用和速度收益这几条关键链路的可行性；下一步可以围绕高质量资产、真实用户反馈和多端协同继续扩展。

尤其是在教育场景中，很多知识点都需要空间结构理解，如果教师能够把日常物体、实验器材和课程模型逐步沉淀为 3D 素材库，课堂内容会变得更直观，也更容易复用。

在技术上，后续可以引入更高质量的 image-to-3D 后端作为可替换模块，让系统在需要时生成更精细的展示模型；在产品上，可以增加教师备课模式、学生实践模式、素材收藏和共享机制；在数据上，可以持续收集 auto_hit、review 和 miss 的真实日志，让规则策略逐步过渡到更稳健的轻量学习策略。整个演进方向仍然围绕一个核心目标：用可解释、可回退、可复用的方式降低 3D 内容生成和展示的门槛。

未来方向	具体能力	预期价值
高质量生成后端	接入云端或 Linux 环境下更稳定的高质量 3D 生成模型	提升展示模型精细度
Review 第二阶段	允许用户选择复用或继续生成，并记录选择	让边界样本也能产生速度收益
素材库管理	按课程、类别、来源、许可管理 GLB	让资产长期可维护
多用户共享	局域网或云端缓存共享	降低团队重复生成成本
轻量学习策略	在样本增加后评估 Logistic Regression、RandomForest 或 MLP	提高复杂样本下的判断稳定性
教学场景模板	实验器材、机械结构、生活物体、科普展项模板	帮助教师快速组织 AR 展示内容

11.10 应用延展与推广设想

从应用延展角度看，造物π可以落在多个具体方向：面向学校，可以作为 AI + AR 创新教学工具；面向实验室，可以作为低门槛三维展示和设备讲解工具；面向科普活动，可以作为观众参与式互动装置；面向创客比赛，可以作为项目路演中的视觉增强组件。它的优势在于无需用户提前准备复杂设备，只要有浏览器、摄像头和模型资产，就能完成基本演示。

推广阶段	目标场景	核心动作	预期效果
校内试点	课程展示、实验课、创新实践课	沉淀 20 到 50 个常用 GLB 资产	形成基础素材库和稳定演示流程
团队共创	学生项目组、实验室、创客社团	共享缓存、记录复用日志、整理演示模板	验证多用户缓存复用价值
平台扩展	跨课程、跨班级、跨活动	接入账号、分类、统计和高质量生成后端	形成持续增长的 AR 3D 素材平台

11.11 评委视角下的产品亮点提炼

从评审视角看，本项目的亮点不只在于某一个模型或某一个页面，而在于把“可交互、可展示、可复用、可验证”四件事情连在了一起。可交互体现为手势与 ROI 框选；可展示体现为浏览器端 GLB 加载和 AR 化呈

现；可复用体现为图文双相似度缓存策略；可验证体现为离线实验、速度收益、三分支验证和多用户仿真。这个结构让作品既有可看的前端效果，也有可解释的技术依据。

评审关注点	本项目回应	可展示材料
创新性	将手势交互、AR 展示和图文双相似度缓存复用结合	技术链路图、策略对比、演示视频
完整性	从摄像头输入到 GLB 展示形成闭环	可运行 Demo、运行手册、页面截图
实用性	减少相似物体重复生成等待，适合教学和展示	auto_hit 速度验证、产品方案章节
安全性	低相似样本进入 miss，边界样本进入 review	三分支验证、Review UI 验证
可扩展性	生成后端、模型资产库、多用户共享均可继续扩展	未来展望、后端替换计划

11.12 演示稳定性与风险控制设计

比赛现场最重要的是稳定。我们把演示风险拆成摄像头风险、生成耗时风险、模型质量风险、网络访问风险和策略误判风险，并分别设计了兜底路径。摄像头不稳定时可以使用录屏片段补充；实时生成耗时较长时使用预置 GLB 和缓存 GLB 展示；模型质量不足时用高质量展示资产承担视觉效果；策略误判风险通过 auto_hit / review / miss 三分支控制。

风险类型	可能表现	当前处理方式	展示口径
摄像头风险	现场设备权限或光照不稳定	提前录制手势与 ROI 片段	保留真实交互流程说明
生成耗时风险	现场等待 3D 生成时间过长	使用缓存 GLB 和预置 GLB 展示	强调缓存复用减少重复等待
模型质量风险	baseline 生成模型粗糙	比赛展示使用可看的 GLB 资产	说明生成后端可替换，主线是缓存复用
网络访问风险	固定链接或隧道偶发不可用	本地服务和录屏作为兜底	保证视频和文档可完整说明
策略误判风险	相似但不确定样本可能被错误复用	review 区只提示，低相似样本 miss	展示保守、可回退的工程设计

11.13 从比赛作品到真实应用的推进计划

如果继续推进，本项目可以按“稳定演示、真实试用、平台沉淀”三步走。稳定演示阶段以比赛交付为目标，确保页面可打开、视频可说明、模型可展示；真实试用阶段选择一个班级、实验室或活动场景，收集 3 到 5 类常见物体的使用日志；平台沉淀阶段将常用模型资产、复用记录 and 用户反馈整理成可持续发展的素材库。

推进阶段	主要任务	关键指标	预期成果
------	------	------	------

稳定演示	完善固定 Demo、讲解视频、说明文档和展示资产	页面可访问、演示无阻塞、素材清晰	完成比赛提交与现场展示
真实试用	在小范围课堂或团队中使用 3 到 5 个物体类别	复用次数、等待时间、用户反馈	验证系统在真实场景中的使用价值
素材沉淀	整理 GLB 来源、类别、许可和复用记录	资产数量、复用率、错误复用次数	形成课程或活动专用模型库
能力升级	完善 Review 选择、接入高质量生成后端、扩充样本	模型质量、策略稳定性、平均节省时间	从演示工具走向可持续平台

总体来看，造物π的未来方向不是把所有能力一次性做重，而是保持“轻量入口 + 资产复用 + 可替换后端”的架构。这样既能支撑当前比赛展示，也能为后续课程化、团队化和平台化留下清晰路径。

十二、源代码与可复现性说明

本项目说明文档采用核心模块说明与源码查看页面结合的方式呈现。完整源码以可查看的源码说明页面和项目文件形式提供，正文只保留核心模块说明、运行方式和可复现路径，便于评委快速理解系统结构，同时保持说明文档正文简洁清晰。

12.1 核心代码模块

模块	文件或目录	功能	对应项目能力
Web 主程序	plus.py	负责 Flask 服务、摄像头页面、SSE 事件、前端模型展示和缓存策略接入	系统主入口、AR 页面、缓存复用展示
缓存策略加载	cache_policy_loader.py	加载图文双相似度策略，计算 policy_score 并输出 auto_hit / review / miss	缓存复用决策
策略 dry-run	cache_policy_dry_run.py	在不启动完整生成流程的情况下验证策略判断	工程验证
相似度对比	make_similarity_method_comparison.py	整理 text-only、image-only、rule-fusion、dual-threshold fusion 对比	方法验证
多用户仿真	simulate_multi_user_cache_reuse.py	模拟共享缓存、网络延迟和节省时间	多用户协同缓存验证
比赛交付物	paper_repro_outputs/competition_delivery/	保存说明文档、视频脚本、资产清单和交付状态	比赛提交材料

12.2 可运行 Demo 链路

比赛 Demo 的最小运行链路为：启动 Flask 服务，打开固定浏览器地址，使用摄像头输入和 MediaPipe 手势识别完成 ROI 选择，随后加载预置 GLB 或缓存 GLB，并展示 auto_hit、review、miss 三类策略效果。比赛演示以缓存 GLB 与预置 GLB 为主要展示路径，现场 TripoSR 生成作为讲解对比内容。

- 固定浏览器地址：<https://zaowupi-ar3d-demo.serveusercontent.com>
- 推荐启动命令：设置 PYTHONIOENCODING、ENABLE_POLICY_CACHE 和 CACHE_POLICY_PATH 后运行 `.\venv\Scripts\python.exe plus.py`

12.3 可复现实验链路

当前实验结果可以通过已有脚本和结果目录复核，主要包括 v3_real_70 样本实验、相似度方法对比、auto_hit 速度收益验证、miss 安全回退验证和多用户缓存复用仿真。相关结果保存在 paper_repro_outputs/cache_similarity_eval_v3_real_70/ 目录下。

- 相似度方法对比: paper_repro_outputs/cache_similarity_eval_v3_real_70/similarity_method_comparison/
- 速度收益验证: paper_repro_outputs/cache_similarity_eval_v3_real_70/speed_runtime_validation/
- 多用户缓存仿真: paper_repro_outputs/cache_similarity_eval_v3_real_70/multi_user_cache_simulation/

12.4 源码查看说明

源码与复现说明已整理为可直接打开的 HTML 页面：
paper_repro_outputs/competition_delivery/source_view/source_and_reproducibility.html。该页面用于说明核心源码结构、运行链路和实验复现路径，适合随比赛材料一并提交或展示。

十三、比赛提交材料对应关系

为了让评委能够快速对应不同材料的作用，我们将比赛交付物整理为“可体验、可观看、可阅读、可复核”四类。固定 Demo 地址用于体验系统入口，演示视频用于观看完整流程，说明文档用于理解技术与产品价值，源码说明页面用于查看工程结构和复现路径。四类材料共同构成作品闭环。

提交材料	对应内容	评委可看到的价值
Demo 链接	固定浏览器地址、摄像头页面、模型展示和缓存策略提示	证明系统具备可访问、可运行的交互入口
演示视频	手势识别、ROI 框选、GLB 展示、auto_hit 加速、review 提示	在不受现场环境影响的情况下完整理解流程
说明文档	背景、架构、技术链路、实验数据、产品方案和未来展望	理解项目创新点、落地价值和扩展空间
源码说明页面	核心模块、运行命令、实验目录和复现路径	查看工程结构，确认作品具备可复核性
实验结果目录	相似度对比、速度验证、多用户仿真、策略报告	支撑缓存复用提速和安全回退的技术结论

这套交付结构的重点是让作品既能被直接体验，也能被完整解释。即使现场网络或摄像头环境出现波动，评委仍然可以通过视频、PDF 和源码说明页面理解系统链路；如果现场环境正常，则可以通过固定 Demo 地址体验交互页面和模型展示效果。

造物π项目将 AI 多模态感知、AR 交互、3D 模型展示和缓存复用优化结合在一起。我们通过 MediaPipe 手势姿态估计实现自然交互，通过 ROI 语义识别理解目标物体，通过图文双相似度缓存复用减少重复 3D 生成，从而把课堂中的现实物体转化为更易展示和复用的数字素材。

- 后续接入更高质量的 3D 生成后端，让展示模型更适合课堂和产品演示。
- 完善 Review UI 交互，让用户可以在边界情况下选择是否复用缓存。
- 做真实多用户协同缓存实验，验证局域网中共享模型缓存是否能稳定减少等待时间。
- 样本继续扩充后，再考虑 Logistic Regression、RandomForest 或 MLP 等学习型判断方法。

十四、总结与后续工作

附录：核心源代码

本附录收录项目核心源码文件，方便评委直接查看系统主入口、缓存策略、离线验证和实验脚本。附录仅包含核心模块；虚拟环境、临时缓存、模型权重和个人配置不纳入正文。

Web 主程序：plus.py

文件路径：D:\tool3\py1\pythonProject6\plus.py

代码行数：1288

```
0001     """
0002 AR 手势阅读助手 - 开源模型版 v3.6.1
0003
0004 目标：
0005 1. 保留原来的 Flask + MediaPipe + SSE + 前端 AR 交互结构。
0006 2. 将图像理解从智谱 GLM-4V 替换为本地开源 Qwen2.5-VL。
0007 3. 将 3D 生成从 TokenHub 替换为本地开源 TripoSR CLI。
0008 4. 保留本地 3D 模型缓存，重复物体直接加载缓存。
0009
0010 推荐先单独跑通：
0011     Qwen2.5-VL：本地图像理解
0012     TripoSR：python run.py input.png --output-dir output --model-save-format glb
0013
0014 运行前建议配置环境变量：
0015 PowerShell 示例：
0016     $env:HAND_MODEL_PATH="D:\\tool3\\py1\\pythonProject6\\hand_landmarker.task"
0017     $env:LOCAL_VLM_MODEL="Qwen/Qwen2.5-VL-3B-Instruct"
0018     $env:TRIPOSR_DIR="D:\\tool3\\TripoSR"
0019     $env:TRIPOSR_PYTHON="D:\\tool3\\TripoSR\\venv\\Scripts\\python.exe"
0020     $env:LOCAL_3D_DEVICE="cuda:0"
0021     python ar_gesture_reader_open_source_v3_6_1_fixed.py
0022
0023 如果机器显存不够，可先设置：
0024     $env:LOCAL_VLM_MODEL="Qwen/Qwen2.5-VL-3B-Instruct"
0025     $env:LOCAL_3D_DEVICE="cpu"
0026
0027 注意：
0028 - Qwen2.5-VL 和 TripoSR 第一次运行会下载或加载模型，耗时较长。
0029 - TripoSR 生成质量依赖 ROI 清晰度，最好框选单个物体，背景尽量简单。
0030 """
0031
0032 from __future__ import annotations
0033
0034 import asyncio
0035 import collections
0036 import hashlib
0037 import json
0038 import logging
0039 import math
0040 import os
0041 import queue
0042 import re
0043 import shutil
0044 import subprocess
0045 import threading
0046 import time
0047 import uuid
0048 from dataclasses import dataclass
0049 from pathlib import Path
0050 from typing import Any, Deque, Dict, List, Optional, Tuple
0051
0052 import cv2
0053 import edge_tts
0054 import mediapipe as mp
0055 import numpy as np
0056 import requests
0057 from flask import Flask, Response, send_from_directory
0058 from mediapipe import tasks
0059 from mediapipe.tasks.python import vision
0060
0061
0062 # ===== 日志 =====
0063
0064 logging.basicConfig(level=logging.INFO, format="[%(asctime)s] %(levelname)s - %(message)s")
0065 logger = logging.getLogger("ar-gesture-reader-open-source")
0066
0067 try:
0068     from cache_policy_loader import decide_cache_policy, load_cache_policy
```

```

0069     _POLICY_LOADER_AVAILABLE = True
0070     _POLICY_LOADER_IMPORT_ERROR: Optional[Exception] = None
0071 except Exception as exc:
0072     decide_cache_policy = None # type: ignore[assignment]
0073     load_cache_policy = None # type: ignore[assignment]
0074     _POLICY_LOADER_AVAILABLE = False
0075     _POLICY_LOADER_IMPORT_ERROR = exc
0076
0077
0078 # ===== 基础配置 =====
0079
0080 @dataclass(frozen=True)
0081 class Config:
0082     model_path: str = os.getenv("HAND_MODEL_PATH", r"D:\tool3\py1\pythonProject6\hand_landmarker.task")
0083
0084     camera_index: int = int(os.getenv("CAMERA_INDEX", "0"))
0085     camera_width: int = int(os.getenv("CAMERA_WIDTH", "640"))
0086     camera_height: int = int(os.getenv("CAMERA_HEIGHT", "480"))
0087     jpeg_quality: int = int(os.getenv("JPEG_QUALITY", "85"))
0088
0089     trigger_frames: int = int(os.getenv("TRIGGER_FRAMES", "15"))
0090     cooldown_seconds: float = float(os.getenv("COOLDOWN_SECONDS", "5"))
0091     pinch_threshold_norm: float = float(os.getenv("PINCH_THRESHOLD_NORM", "0.07"))
0092     min_roi_area_ratio: float = float(os.getenv("MIN_ROI_AREA_RATIO", "0.02"))
0093     gesture_event_interval: float = float(os.getenv("GESTURE_EVENT_INTERVAL", "0.35"))
0094     roi_multiframe_count: int = int(os.getenv("ROI_MULTIFRAME_COUNT", "12"))
0095     enable_roi_keyframe_preprocess: bool = os.getenv("ENABLE_ROI_KEYFRAME_PREPROCESS", "1") == "1"
0096
0097     static_model_dir: Path = Path(os.getenv("STATIC_MODEL_DIR", "runtime_assets"))
0098
0099     # 本地图像理解模型: Qwen2.5-VL
0100     local_vlm_model: str = os.getenv(
0101         "LOCAL_VLM_MODEL",
0102         r"D:\tool3\models\Qwen2.5-VL-3B-Instruct",
0103     )
0104     local_vlm_max_new_tokens: int = int(os.getenv("LOCAL_VLM_MAX_NEW_TOKENS", "160"))
0105     local_vlm_4bit: bool = os.getenv("LOCAL_VLM_4BIT", "0") == "1"
0106
0107     # 本地 3D 生成: TriposR
0108     triposr_dir: str = os.getenv("TRIPOSR_DIR", r"D:\tool3\TriposR")
0109     triposr_python: str = os.getenv("TRIPOSR_PYTHON", r"D:\tool3\TriposR\venv\Scripts\python.exe")
0110     local_3d_device: str = os.getenv(
0111         "LOCAL_3D_DEVICE",
0112         "cpu",
0113     )
0114     local_3d_timeout: int = int(os.getenv("LOCAL_3D_TIMEOUT", "600"))
0115     triposr_bake_texture: bool = os.getenv("TRIPOSR_BAKE_TEXTURE", "0") == "1"
0116     triposr_texture_resolution: int = int(os.getenv("TRIPOSR_TEXTURE_RESOLUTION", "1024"))
0117     enable_geometry_texture_split: bool = os.getenv("ENABLE_GEOMETRY_TEXTURE_SPLIT", "1") == "1"
0118     enable_progressive_model_loading: bool = os.getenv("ENABLE_PROGRESSIVE_MODEL_LOADING", "1") == "1"
0119     enable_policy_cache: bool = os.getenv("ENABLE_POLICY_CACHE", "0") == "1"
0120     cache_policy_path: str = os.getenv("CACHE_POLICY_PATH", "runtime_assets/cache_policy.json")
0121
0122
0123 CONFIG = Config()
0124 CONFIG.static_model_dir.mkdir(parents=True, exist_ok=True)
0125 logger.info(
0126     "policy_cache_enabled=%s, cache_policy_path=%s",
0127     CONFIG.enable_policy_cache,
0128     CONFIG.cache_policy_path,
0129 )
0130
0131 MODEL_CACHE_DIR = CONFIG.static_model_dir / "model_cache"
0132 MODEL_CACHE_DIR.mkdir(parents=True, exist_ok=True)
0133 CACHE_INDEX_PATH = MODEL_CACHE_DIR / "cache_index.json"
0134 MODEL_SPLIT_DIR = MODEL_CACHE_DIR / "split_artifacts"
0135 MODEL_SPLIT_DIR.mkdir(parents=True, exist_ok=True)
0136
0137 LOCAL_3D_WORK_DIR = CONFIG.static_model_dir / "local_3d_work"
0138 LOCAL_3D_WORK_DIR.mkdir(parents=True, exist_ok=True)
0139
0140 ROI_SEQUENCE_DIR = CONFIG.static_model_dir / "roi_sequences"
0141 ROI_SEQUENCE_DIR.mkdir(parents=True, exist_ok=True)
0142
0143 if CONFIG.enable_policy_cache and not _POLICY_LOADER_AVAILABLE:
0144     logger.warning("策略缓存已请求启用, 但 cache_policy_loader 导入失败, 已自动禁用: %s", _POLICY_LOADER_IMPORT_ERROR)
0145
0146
0147 # ===== Flask 和全局状态 =====
0148
0149 app = Flask(__name__)
0150 sse_queue: "queue.Queue[Dict[str, Any]]" = queue.Queue(maxsize=200)
0151 processing_lock = threading.Lock()
0152
0153 # Qwen 模型懒加载缓存
0154 _QWEN_MODEL = None
0155 _QWEN_PROCESSOR = None
0156 _QWEN_LOCK = threading.Lock()
0157
0158

```



```
0159 # ===== 前端页面 =====
0160
0161 HTML_PAGE = r'''<!DOCTYPE html>
0162 <html lang="zh-CN">
0163 <head>
0164 <meta charset="UTF-8">
0165 <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no">
0166 <title>AR 手势阅读助手 - 开源模型版</title>
0167 <script type="module" src="https://unpkg.com/@google/model-viewer/dist/model-viewer.min.js"></script>
0168 <style>
0169 *{margin:0;padding:0;box-sizing:border-box}html,body{width:100%;height:100%;overflow:hidden;background:#000;font-family:Arial,"Microsoft
YaHei",sans-serif}#container{position:relative;width:100vw;height:100vh;overflow:hidden;background:#000}#video-
feed{width:100%;height:100%;object-fit:cover}@media(min-width:768px){#video-feed{width:auto;height:auto;max-width:min(100%,900px);max-
height:min(100%,680px);position:absolute;top:50%;left:50%;transform:translate(-50%,-50%);object-fit:contain;background:#000;border-
radius:12px}}#ar-model-layer{position:absolute;left:50%;top:50%;width:260px;height:260px;transform:translate(-50%,-50%) scale(1)
rotate(0deg);z-index:18;display:none;pointer-events:auto;touch-action:none}#ar-model-layer.show{display:block}#ar-model-container,#ar-model-
container model-viewer{width:100%;height:100%;background:transparent}#status-bar{position:absolute;top:10px;left:50%;transform:translateX(-
50%);background:rgba(0,0,0,.64);color:#fff;padding:7px 18px;border-radius:20px;font-size:14px;pointer-events:none;z-index:30;backdrop-
filter:blur(4px);max-width:92vw;white-space:nowrap;overflow:hidden;text-overflow:ellipsis}#gesture-
popup{position:absolute;bottom:84px;left:50%;transform:translateX(-50%);background:rgba(0,0,0,.78);color:#fff;padding:12px 24px;border-
radius:30px;font-size:16px;font-weight:bold;pointer-events:none;z-index:30;opacity:0;transition:opacity .25s;backdrop-
filter:blur(4px)}#gesture-popup.show{opacity:1}#btn-group{position:absolute;bottom:20px;left:50%;transform:translateX(-
50%);display:flex;gap:10px;z-index:40;flex-wrap:wrap;justify-content:center}#btn-group button{padding:10px 18px;border:none;border-
radius:25px;background:rgba(255,255,255,.2);color:#fff;font-size:14px;cursor:pointer;backdrop-filter:blur(4px);transition:background .2s}#btn-
group button:active,#btn-group button.touch-active{background:rgba(255,255,255,.45)}#info-
card{position:absolute;right:14px;top:58px;width:min(350px,calc(100vw - 28px));max-height:calc(100vh -
150px);overflow:auto;background:rgba(0,0,0,.62);color:#fff;border:1px solid rgba(255,255,255,.16);border-radius:16px;padding:12px;z-
index:25;backdrop-filter:blur(6px);display:none}#info-card.show{display:block}#knowledge{line-height:1.55;font-size:14px;margin-
bottom:10px;white-space:pre-wrap}#model-link{display:none;color:#9ad1ff;font-size:13px;margin-top:8px;word-break:break-all}#model-
link.show{display:block}#hint{margin-top:8px;font-size:12px;line-height:1.45;color:rgba(255,255,255,.72)}#badge{display:inline-block;margin-
top:8px;padding:3px 8px;border-radius:999px;background:rgba(80,180,255,.18);border:1px solid rgba(120,210,255,.25);font-
size:12px;color:#c9ffff}#review-card{position:absolute;left:14px;top:58px;width:min(340px,calc(100vw -
28px));background:rgba(8,14,18,.78);color:#fff;border:1px solid rgba(120,210,255,.26);border-radius:12px;padding:12px;z-index:28;backdrop-
filter:blur(6px);display:none;box-shadow:0 8px 24px rgba(0,0,0,.22)}#review-card.show{display:block}.review-title{font-size:15px;font-
weight:bold;margin-bottom:6px}.review-body{font-size:13px;line-height:1.5;color:rgba(255,255,255,.86)}.review-meta{margin-top:8px;font-
size:12px;color:#c9ffff}.review-actions{display:flex;gap:8px;margin-top:10px;flex-wrap:wrap}.review-actions button{border:1px solid
rgba(255,255,255,.18);border-radius:8px;padding:7px 10px;background:rgba(255,255,255,.08);color:rgba(255,255,255,.62)}
0170 </style>
0171 </head>
0172 <body>
0173 <div id="container">
0174 
0175 <div id="ar-model-layer"><div id="ar-model-container"></div></div>
0176 <div id="status-bar">👁 等待手势...</div>
0177 <div id="gesture-popup"></div>
0178 <div id="info-card">
0179 <div id="knowledge">等待 AI 识别结果...</div>
0180 <a id="model-link" target="_blank" rel="noopener">打开 3D 模型文件</a>
0181 <div id="badge">开源模型版: Qwen2.5-VL + TripoSR</div>
0182 <div id="hint">交互提示: 一只手握合拖动模型; 两只手握合缩放模型; 左右滑动旋转模型.</div>
0183 </div>
0184 <div id="review-card" aria-live="polite">
0185 <div class="review-title">发现相似缓存模型</div>
0186 <div id="review-body" class="review-body">系统检测到一个相似缓存模型, 后续可支持一键复用。当前实验阶段仍将继续原生成流程.</div>
0187 <div id="review-meta" class="review-meta"></div>
0188 <div class="review-actions"><button disabled>复用缓存模型 (后续开放)</button><button disabled>继续生成</button></div>
0189 </div>
0190 <div id="btn-group">
0191 <button id="btn-scroll-up">⬆ 上滚</button><button id="btn-scroll-down">⬇ 下滚</button><button id="btn-zoom-
in">🔍 +</button><button id="btn-zoom-out">🔍 -</button>
0192 </div>
0193 </div>
0194 <script>
0195 const gesturePopup=document.getElementById('gesture-popup');const statusBar=document.getElementById('status-bar');const
infoCard=document.getElementById('info-card');const knowledge=document.getElementById('knowledge');const
modelLink=document.getElementById('model-link');const arModelLayer=document.getElementById('ar-model-layer');const
arModelContainer=document.getElementById('ar-model-container');const reviewCard=document.getElementById('review-card');const
reviewBody=document.getElementById('review-body');const reviewMeta=document.getElementById('review-meta');let
popupTimer=null,modelScale=1,modelX=.5,modelY=.5,modelRotationY=0,modelLoaded=false;const eventSource=new
EventSource('/events');eventSource.onmessage=function(e){try{handleServerEvent(JSON.parse(e.data))}catch(err){console.error('SSE JSON parse
error:',err,e.data)}};eventSource.onerror=function(){statusBar.textContent='⚠ SSE 连接中断, 正在尝试重连...'};function
handleServerEvent(data){switch(data.type){case'gesture':handleGesture(data);break;case'drag':handleDrag(data);break;case'scale':handleScale(dat
a);break;case'status':setStatus(data.text||'处理中...');showPopup(data.text||'处理
中...');break;case'knowledge':showKnowledge(data.text||'');break;case'audio':playAudio(data.url);break;case'model':showModel(data.url,data);bre
ak;case'cache_review':showCacheReview(data);break;case'error':setStatus('✖ '+data.text||'发生错误');showPopup(data.text||'发生错误
');break;default:console.debug('未处理事件:',data)}}function showCacheReview(data){if(!reviewCard)return;const score=data.policy_score==null?'-
':Number(data.policy_score).toFixed(2);const weak=data.weak_threshold==null?'-':data.weak_threshold;const
strong=data.strong_threshold==null?'-':data.strong_threshold;reviewBody.textContent=data.text||'系统检测到一个相似缓存模型, 后续可支持一键复用。当前
实验阶段仍将继续原生成流程.';reviewMeta.textContent=`候选: ${data.best_keyword||'未知'} | score:${score} | weak:${weak} |
strong:${strong}`;reviewCard.classList.add('show');setStatus('发现相似缓存模型, 当前仍继续原生成流程');showPopup('发现相似缓存模型, 后续可支持复用
');function setStatus(text){statusBar.textContent=text}function handleGesture(data){const
gesture=data.gesture,detail=data.detail,confidence=data.confidence;const percent=confidence?' (${Math.round(confidence*100)}%)':'';let
text='';switch(gesture){case'open_palm':text='👐 张开手掌 - 待命中';break;case'point_up':text='👉 食指上指 - 向上滚动
';break;case'point_down':text='👇 食指下指 - 向下滚动';break;case'fist':text='👊 握拳 - 确认/选中';break;case'two_fingers_up':text='👉 两指上指 -
放大';break;case'two_fingers_down':text='👇 两指下指 - 缩小';break;case'swipe_left':text='👉 左滑 - 旋转模型';break;case'swipe_right':text='👈 右滑 -
旋转模型';break;default:text='👋 未识别手势'}setStatus(text+percent);if(detail)showPopup(detail);executeAction(gesture)}function
showPopup(msg){if(!msg)return;if(popupTimer)clearTimeout(popupTimer);gesturePopup.textContent=msg;gesturePopup.classList.add('show');popupTimer
```

```

=setTimeout(()=>gesturePopup.classList.remove('show'),1100)}function executeAction(gesture){window.parent.postMessage({source:'ar-gesture-
reader',gesture:gesture},'*');window.dispatchEvent(new CustomEvent('ar-
gesture',{detail:{gesture:gesture}}));switch(gesture){case'point_up':window.scrollBy({top:-
120,behavior:'smooth'});break;case'point_down':window.scrollBy({top:120,behavior:'smooth'});break;case'two_fingers_up':handleScale({factor:1.08}
);break;case'two_fingers_down':handleScale({factor:.92});break;case'swipe_left':rotateModel(-
20);break;case'swipe_right':rotateModel(20);break}}function
handleDrag(data){if(!data.active||!modelLoaded)return;modelX=Number(data.x||modelX);modelY=Number(data.y||modelY);updateArModelTransform();wind
ow.dispatchEvent(new CustomEvent('ar-drag',{detail:data}))}function handleScale(data){if(!modelLoaded)return;const
factor=Number(data.factor||1);modelScale=Math.min(3.2,Math.max(.35,modelScale*factor));updateArModelTransform();const
viewer=arModelContainer.querySelector('model-viewer');if(viewer)viewer.setAttribute('scale','${modelScale} ${modelScale}
${modelScale}');window.dispatchEvent(new CustomEvent('ar-scale',{detail:{factor:factor,modelScale:modelScale}}))}function
rotateModel(delta){if(!modelLoaded)return;modelRotationY+=delta;updateArModelTransform()}function
updateArModelTransform(){if(!arModelLayer)return;const
x=modelX*window.innerWidth,y=modelY*window.innerHeight;arModelLayer.style.left=`${x}px`;arModelLayer.style.top=`${y}px`;arModelLayer.style.trans
form=`translate(-50%,-50%) scale(${modelScale}) rotateY(${modelRotationY}deg)`}function
showKnowledge(text){infoCard.classList.add('show');knowledge.textContent=text||'没有识别到有效内容'}function playAudio(url){if(!url)return;const
audio=new Audio(url+'?t='+Date.now());audio.play().catch(err=>console.warn('浏览器阻止自动播放,需要用户先点击页面:',err))}function
showModel(url,meta={}){if(!url)return;infoCard.classList.add('show');modelLink.href=url;modelLink.textContent='打开 3D 模型文件
';modelLink.classList.add('show');arModelLayer.classList.add('show');const started=performance.now();arModelContainer.innerHTML=`<model-viewer
src="${url}"?t=${Date.now()}>`camera-controls auto-rotate ar exposure="1" shadow-intensity="0.7" interaction-prompt="none"></model-
viewer>`;const viewer=arModelContainer.querySelector('model-viewer');if(viewer){viewer.addEventListener('load',()=>{const
ms=Math.round(performance.now()-started);console.log('模型加载完
成:',{load_ms:ms,cache_hit:meta.cache_hit||false,keyword:meta.keyword||'',url:url}},{once:true});modelX=.5;modelY=.5;modelScale=1;modelRotatio
nY=0;modelLoaded=true;updateArModelTransform();if(meta.cache_hit){setStatus('✔ 已从缓存加载 3D 模型');showPopup('缓存命中,模型已快速加载
')}}else{setStatus('✔ 3D 模型已出现在画面中');showPopup('3D 模型已出现,可用手势互动')}}function
showModel(url,meta={}){if(!url)return;infoCard.classList.add('show');const geometryUrl=meta.geometry_url||'';const
fullUrl=meta.full_url||url;const firstUrl=geometryUrl||fullUrl;modelLink.href=fullUrl;modelLink.textContent='打开 3D 模型文件
';modelLink.classList.add('show');arModelLayer.classList.add('show');modelX=.5;modelY=.5;modelScale=1;modelRotationY=0;modelLoaded=true;const
started=performance.now();let geometryVisibleMs=null;function mountModel(src,phase,onLoaded){arModelContainer.innerHTML=`<model-viewer
src="${src}"?t=${Date.now()}>`camera-controls auto-rotate ar exposure="1" shadow-intensity="0.7" interaction-prompt="none"></model-
viewer>`;const viewer=arModelContainer.querySelector('model-viewer');if(viewer){viewer.addEventListener('load',()=>{const
ms=Math.round(performance.now()-started);console.log('模型加载阶
段:',{phase:phase,load_ms:ms,geometry_first_visible_ms:geometryVisibleMs,cache_hit:meta.cache_hit||false,keyword:meta.keyword||'',url:src});if(
onLoaded)onLoaded(ms)},{once:true});updateArModelTransform()}if(geometryUrl&&geometryUrl!=fullUrl){mountModel(geometryUrl,'geometry',ms=>{geom
etryVisibleMs=ms;setStatus('✔ 几何模型已先显示,正在加载完整外观');setTimeout(()=>mountModel(fullUrl,'full',()=>{setStatus('✔ 完整 3D 模型已出现
'});showPopup('完整模型已替换
'}),60)}}else{mountModel(firstUrl,'full',null)}updateArModelTransform();if(meta.cache_hit){setStatus(geometryUrl?'✔ 缓存命中,先显示几何模型
':'✔ 已从缓存加载 3D 模型');showPopup('缓存命中,模型已快速加载')}}else{setStatus(geometryUrl?'✔ 3D 几何模型先行显示':'✔ 3D 模型已出现在画面中
'});showPopup('3D 模型已出现,可用手势互动')}}function addButtonListeners(id,gestureName){const
btn=document.getElementById(id);if(!btn)return;[ 'mousedown', 'touchstart' ].forEach(evt=>btn.addEventListener(evt,function(e){e.preventDefault()
});btn.classList.add('touch-
active');executeAction(gestureName)});[ 'mouseup', 'mouseleave', 'touchend', 'touchcancel' ].forEach(evt=>btn.addEventListener(evt,function(){btn.cl
asslist.remove('touch-active')}));addButtonListeners('btn-scroll-up','point_up');addButtonListeners('btn-scroll-
down','point_down');addButtonListeners('btn-zoom-in','two_fingers_up');addButtonListeners('btn-zoom-
out','two_fingers_down');window.addEventListener('resize',updateArModelTransform);console.log('✔ AR 手势阅读助手开源模型版前端已就绪');
0196 </script>
0197 </body></html>
0198 '''
0199
0200
0201 # ===== 通用工具函数 =====
0202
0203 def push_event(event: Dict[str, Any]) -> None:
0204     try:
0205         sse_queue.put_nowait(event)
0206     except queue.Full:
0207         try:
0208             sse_queue.get_nowait()
0209         except queue.Empty:
0210             pass
0211         try:
0212             sse_queue.put_nowait(event)
0213     except queue.Full:
0214         logger.warning("SSE 队列已满,丢弃事件: %s", event.get("type"))
0215
0216
0217 def sse_pack(event: Dict[str, Any]) -> str:
0218     return f"data: {json.dumps(event, ensure_ascii=False)}\n\n"
0219
0220
0221 def make_message_jpeg(message: str, width: int = 640, height: int = 480) -> bytes:
0222     frame = np.full((height, width, 3), 20, dtype=np.uint8)
0223     cv2.putText(frame, message[:42], (30, height // 2), cv2.FONT_HERSHEY_SIMPLEX, 0.75, (255, 255, 255), 2)
0224     ok, jpeg = cv2.imencode(".jpg", frame, [cv2.IMWRITE_JPEG_QUALITY, 80])
0225     return jpeg.tobytes() if ok else b""
0226
0227
0228 def dist_norm(a: Any, b: Any) -> float:
0229     return math.hypot(a.x - b.x, a.y - b.y)
0230
0231
0232 def dist_px(a: Tuple[float, float], b: Tuple[float, float]) -> float:
0233     return math.hypot(a[0] - b[0], a[1] - b[1])
0234
0235
0236 HAND_LINES: List[Tuple[int, int]] = [
0237     (0, 1), (1, 2), (2, 3), (3, 4),
0238     (0, 5), (5, 6), (6, 7), (7, 8),
0239     (0, 9), (9, 10), (10, 11), (11, 12),

```

```

0240     (0, 13), (13, 14), (14, 15), (15, 16),
0241     (0, 17), (17, 18), (18, 19), (19, 20),
0242 ]
0243
0244
0245 # ===== 缓存工具 =====
0246
0247 def load_cache_index() -> Dict[str, Dict[str, Any]]:
0248     if not CACHE_INDEX_PATH.exists():
0249         return {}
0250     try:
0251         return json.loads(CACHE_INDEX_PATH.read_text(encoding="utf-8"))
0252     except Exception:
0253         return {}
0254
0255
0256 def save_cache_index(index: Dict[str, Dict[str, Any]]) -> None:
0257     CACHE_INDEX_PATH.write_text(json.dumps(index, ensure_ascii=False, indent=2), encoding="utf-8")
0258
0259
0260 def normalize_keyword(keyword: str) -> str:
0261     keyword = (keyword or "").strip()
0262     for word in ["一个", "一只", "一台", "可爱的", "有趣的", "卡通", "模型", "玩具", "小朋友", "儿童", "物体"]:
0263         keyword = keyword.replace(word, "")
0264     keyword = keyword.strip()
0265     aliases = {
0266         "小恐龙": "恐龙", "恐龙玩偶": "恐龙", "恐龙玩具": "恐龙",
0267         "地球仪模型": "地球仪", "玩具熊": "熊", "泰迪熊": "熊",
0268         "小汽车": "汽车", "汽车玩具": "汽车", "火箭模型": "火箭", "飞机模型": "飞机",
0269     }
0270     return aliases.get(keyword, keyword or "未知物体")
0271
0272
0273 def safe_filename(name: str) -> str:
0274     name = normalize_keyword(name)
0275     visible = "".join(c for c in name if c.isalnum() or c in "-_")[:24] or "object"
0276     digest = hashlib.md5(name.encode("utf-8")).hexdigest()[:10]
0277     return f"{visible}_{digest}"
0278
0279
0280 def cached_model_path(keyword: str) -> Path:
0281     return MODEL_CACHE_DIR / f"{safe_filename(keyword)}.glb"
0282
0283
0284 def find_cached_model(keyword: str) -> Optional[Path]:
0285     path = cached_model_path(keyword)
0286     return path if path.exists() and path.stat().st_size > 0 else None
0287
0288
0289 def record_cache(keyword: str, model_path: Path, source: str) -> None:
0290     keyword_norm = normalize_keyword(keyword)
0291     index = load_cache_index()
0292     index[keyword_norm] = {
0293         "keyword": keyword_norm,
0294         "filename": model_path.name,
0295         "size_bytes": model_path.stat().st_size if model_path.exists() else 0,
0296         "created_at": time.strftime("%Y-%m-%d %H:%M:%S"),
0297         "source": source,
0298     }
0299     save_cache_index(index)
0300
0301
0302 def split_artifact_dir(keyword: str) -> Path:
0303     return MODEL_SPLIT_DIR / safe_filename(keyword)
0304
0305
0306 def split_report_path(keyword: str) -> Path:
0307     return split_artifact_dir(keyword) / "split_report.json"
0308
0309
0310 def model_url_for_static_path(path: Path) -> str:
0311     rel = Path(path).resolve().relative_to(CONFIG.static_model_dir.resolve())
0312     return "/models/" + rel.as_posix()
0313
0314
0315 def progressive_model_meta(keyword: str, model_path: Path) -> Dict[str, str]:
0316     if not CONFIG.enable_progressive_model_loading:
0317         return {}
0318
0319     artifact_dir = split_artifact_dir(keyword)
0320     geometry_path = artifact_dir / "model_geometry_only.glb"
0321     full_path = artifact_dir / "model_full_textured.glb"
0322
0323     if not geometry_path.exists():
0324         return {}
0325
0326     return {
0327         "geometry_url": model_url_for_static_path(geometry_path),
0328         "full_url": model_url_for_static_path(full_path if full_path.exists() else model_path),
0329         "progressive": True,

```

```

0330     }
0331
0332
0333 def update_cache_split_metadata(keyword: str, split_report_path: Path, split_success: bool) -> None:
0334     keyword_norm = normalize_keyword(keyword)
0335     index = load_cache_index()
0336     item = index.get(keyword_norm, {"keyword": keyword_norm})
0337     item["split_enabled"] = CONFIG.enable_geometry_texture_split
0338     item["split_success"] = split_success
0339     item["split_report"] = str(split_report_path)
0340     item["split_updated_at"] = time.strftime("%Y-%m-%d %H:%M:%S")
0341     index[keyword_norm] = item
0342     save_cache_index(index)
0343
0344
0345 def split_model_for_cache(keyword: str, model_path: Path) -> None:
0346     if not CONFIG.enable_geometry_texture_split:
0347         return
0348
0349     output_dir = split_artifact_dir(keyword)
0350     report_path = split_report_path(keyword)
0351
0352     try:
0353         from paper_geometry_texture_split import split_geometry_texture
0354
0355         report = split_geometry_texture(model_path, output_dir)
0356         update_cache_split_metadata(keyword, report_path, bool(report.success))
0357         if report.success:
0358             logger.info("Geometry/texture split completed: %s", report_path)
0359         else:
0360             logger.warning("Geometry/texture split failed: %s", report.error)
0361     except Exception as exc:
0362         logger.exception("Geometry/texture split post-process failed: %s", exc)
0363         update_cache_split_metadata(keyword, report_path, False)
0364
0365
0366 def start_split_model_for_cache(keyword: str, model_path: Path) -> None:
0367     if not CONFIG.enable_geometry_texture_split:
0368         return
0369
0370     threading.Thread(
0371         target=split_model_for_cache,
0372         args=(keyword, model_path),
0373         daemon=True,
0374     ).start()
0375
0376
0377 def find_fused_cached_model(
0378     keyword: str,
0379     explanation: str,
0380     query_image: Path,
0381     threshold: float = 0.82,
0382 ) -> Optional[Tuple[Path, float, str]]:
0383     try:
0384         from cache_similarity import build_similarity_index, score_cache_entries
0385
0386         index_path = MODEL_CACHE_DIR / "cache_similarity_index.json"
0387         entries = build_similarity_index(
0388             cache_dir=MODEL_CACHE_DIR,
0389             reference_dir=MODEL_CACHE_DIR / "reference_images",
0390             output_path=index_path,
0391         )
0392         query_text = f"{keyword} {explanation}".strip()
0393         results = score_cache_entries(
0394             entries,
0395             query_text=query_text,
0396             query_image=Path(query_image),
0397             threshold=threshold,
0398         )
0399         if not results:
0400             return None
0401
0402         best = results[0]
0403         model_path = Path(best.model_path)
0404         if best.fused_score >= threshold and model_path.exists():
0405             return model_path, float(best.fused_score), best.keyword
0406
0407         return None
0408
0409     except Exception as exc:
0410         logger.warning("融合缓存判断失败, 回退到 TripoSR: %s", exc)
0411         return None
0412
0413
0414 def find_fused_cached_model_with_policy_meta(
0415     keyword: str,
0416     explanation: str,
0417     query_image: Path,
0418 ) -> Optional[Dict[str, Any]]:
0419     try:

```

```

0420         from cache_similarity import build_similarity_index, score_cache_entries
0421
0422         index_path = MODEL_CACHE_DIR / "cache_similarity_index.json"
0423         entries = build_similarity_index(
0424             cache_dir=MODEL_CACHE_DIR,
0425             reference_dir=MODEL_CACHE_DIR / "reference_images",
0426             output_path=index_path,
0427         )
0428         query_text = f"{keyword} {explanation}".strip()
0429         results = score_cache_entries(
0430             entries,
0431             query_text=query_text,
0432             query_image=Path(query_image),
0433             text_weight=0.5,
0434             image_weight=0.5,
0435             threshold=0.7,
0436         )
0437         if not results:
0438             return None
0439
0440         best = results[0]
0441         model_path = Path(best.model_path)
0442         return {
0443             "best_model_path": model_path,
0444             "best_keyword": best.keyword,
0445             "best_filename": best.filename,
0446             "text_score": best.text_score,
0447             "image_score": best.image_score,
0448             "fused_score": best.fused_score,
0449         }
0450     except Exception as exc:
0451         logger.warning("融合缓存 policy metadata 获取失败, 回退原流程: %s", exc)
0452         return None
0453
0454 def decide_policy_cache_from_meta(meta: Dict[str, Any]) -> Tuple[str, Dict[str, Any]]:
0455     info: Dict[str, Any] = {
0456         "policy_cache_enabled": CONFIG.enable_policy_cache,
0457         "fallback_reason": "",
0458         "best_model_path": str(meta.get("best_model_path") or ""),
0459         "text_score": meta.get("text_score"),
0460         "image_score": meta.get("image_score"),
0461     }
0462
0463     if not CONFIG.enable_policy_cache:
0464         info["fallback_reason"] = "policy_cache_disabled"
0465         return "miss", info
0466     if not _POLICY_LOADER_AVAILABLE or load_cache_policy is None or decide_cache_policy is None:
0467         info["fallback_reason"] = "policy_loader_unavailable"
0468         return "miss", info
0469
0470     try:
0471         policy = load_cache_policy(CONFIG.cache_policy_path)
0472         decision = decide_cache_policy(meta.get("text_score"), meta.get("image_score"), policy)
0473         info.update(decision)
0474         return str(decision.get("policy_decision") or "miss"), info
0475     except Exception as exc:
0476         info["fallback_reason"] = f"policy_error: {exc}"
0477         logger.warning("策略缓存判断失败, 回退原流程: %s", exc)
0478         return "miss", info
0479
0480 # ===== 开源图像理解: Qwen2.5-VL =====
0481
0482 def _load_qwen_vl():
0483     """懒加载 Qwen2.5-VL。首次调用会比较慢。"""
0484     global _QWEN_MODEL, _QWEN_PROCESSOR
0485
0486     with _QWEN_LOCK:
0487         if _QWEN_MODEL is not None and _QWEN_PROCESSOR is not None:
0488             return _QWEN_MODEL, _QWEN_PROCESSOR
0489
0490         push_event({"type": "status", "text": "正在加载本地 Qwen2.5-VL 模型..."})
0491         logger.info("加载本地 VLM: %s", CONFIG.local_vlm_model)
0492
0493         try:
0494             import torch
0495             from transformers import AutoProcessor, Qwen2_5_VLForConditionalGeneration
0496
0497             kwargs: Dict[str, Any] = {"device_map": "auto"}
0498             if torch.cuda.is_available():
0499                 kwargs["torch_dtype"] = torch.float16
0500                 kwargs["attn_implementation"] = "sdpa"
0501             else:
0502                 kwargs["torch_dtype"] = torch.float32
0503
0504             if CONFIG.local_vlm_4bit:
0505                 try:
0506                     from transformers import BitsAndBytesConfig
0507                     kwargs["quantization_config"] = BitsAndBytesConfig(load_in_4bit=True)
0508                 except:
0509                     pass

```

```

0510         except Exception as exc:
0511             logger.warning("4bit 量化不可用, 改用普通加载: %s", exc)
0512
0513         _QWEN_MODEL = Qwen2_5_VLForConditionalGeneration.from_pretrained(CONFIG.local_vlm_model, **kwargs)
0514         _QWEN_PROCESSOR = AutoProcessor.from_pretrained(CONFIG.local_vlm_model)
0515         logger.info("Qwen2.5-VL 加载完成")
0516         return _QWEN_MODEL, _QWEN_PROCESSOR
0517
0518     except Exception as exc:
0519         logger.exception("Qwen2.5-VL 加载失败: %s", exc)
0520         raise
0521
0522
0523 def _parse_vlm_output(text: str) -> Tuple[str, str]:
0524     text = (text or "").strip()
0525
0526     # 优先解析 JSON
0527     try:
0528         m = re.search(r"\{.*\}", text, re.S)
0529         if m:
0530             obj = json.loads(m.group(0))
0531             explanation = str(obj.get("explanation") or obj.get("讲解") or "").strip()
0532             keyword = str(obj.get("keyword") or obj.get("关键词") or obj.get("object") or "").strip()
0533             if explanation and keyword:
0534                 return explanation, normalize_keyword(keyword)
0535     except Exception:
0536         pass
0537
0538     # 兼容 “讲解|关键词” 格式
0539     if "|" in text:
0540         explanation, keyword = text.rsplit("|", 1)
0541         return explanation.strip(), normalize_keyword(keyword.strip())
0542
0543     # 兜底: 取前一句作为说明, 关键词使用通用值
0544     return text[:120] or "这是一个有趣的物体。", "可爱的卡通角色"
0545
0546
0547 def analyze_image_local_qwen(image_path: Path) -> str:
0548     """
0549     本地开源图像理解: ROI 图片 -> 儿童化讲解文本 + 关键词。
0550     返回仍保持 explanation|keyword, 方便复用原流程。
0551     """
0552     try:
0553         import torch
0554         from PIL import Image
0555
0556         model, processor = _load_qwen_vl()
0557         image = Image.open(image_path).convert("RGB")
0558
0559         prompt = (
0560             "请观察图片中的主要物体。你是一位幼儿科普老师, 请输出严格 JSON, 不要添加额外文字。"
0561             "JSON 格式为: {\n\"explanation\":\n\"60 字以内的儿童化中文讲解\", \n\"keyword\":\n\"最适合 3D 建模的简短中文物体名\"}。"
0562             "如果看不清, 请给出最可能的类别。"
0563         )
0564
0565         messages = [
0566             {
0567                 "role": "user",
0568                 "content": [
0569                     {"type": "image"},
0570                     {"type": "text", "text": prompt},
0571                 ],
0572             }
0573         ]
0574
0575         text = processor.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
0576         inputs = processor(text=[text], images=[image], padding=True, return_tensors="pt")
0577         inputs = inputs.to(model.device)
0578
0579         with torch.no_grad():
0580             generated_ids = model.generate(**inputs, max_new_tokens=CONFIG.local_vlm_max_new_tokens)
0581
0582             generated_ids_trimmed = [
0583                 out_ids[len(in_ids):] for in_ids, out_ids in zip(inputs.input_ids, generated_ids)
0584             ]
0585             output_text = processor.batch_decode(
0586                 generated_ids_trimmed,
0587                 skip_special_tokens=True,
0588                 clean_up_tokenization_spaces=True,
0589             )[0]
0590
0591             explanation, keyword = _parse_vlm_output(output_text)
0592             logger.info("本地 VLM 输出: %s | %s", explanation, keyword)
0593             return f"{explanation}|{keyword}"
0594
0595     except Exception as exc:
0596         logger.exception("本地 Qwen2.5-VL 图像理解失败: %s", exc)
0597         return "这是一个有趣的物体, 我还没能准确识别它。|可爱的卡通角色"
0598

```



```

0599
0600 # ===== 语音合成 =====
0601
0602 async def tts_to_file(text: str, filename: Path) -> Path:
0603     communicate = edge_tts.Communicate(text, "zh-CN-XiaoxiaoNeural")
0604     await communicate.save(str(filename))
0605     return filename
0606
0607
0608 def speak_and_push(text: str) -> None:
0609     def run() -> None:
0610         filename = CONFIG.static_model_dir / f"tts_{int(time.time())}_{uuid.uuid4().hex[:8]}.mp3"
0611         try:
0612             asyncio.run(tts_to_file(text, filename))
0613             push_event({"type": "audio", "url": f"/models/{filename.name}"})
0614         except Exception as exc:
0615             logger.exception("TTS 生成失败: %s", exc)
0616             push_event({"type": "error", "text": "语音生成失败"})
0617
0618     threading.Thread(target=run, daemon=True).start()
0619
0620
0621 def save_roi_sequence(frames: List[Any]) -> Path:
0622     sequence_dir = ROI_SEQUENCE_DIR / f"roi_seq_{int(time.time())}_{uuid.uuid4().hex[:8]}"
0623     sequence_dir.mkdir(parents=True, exist_ok=True)
0624
0625     for idx, roi in enumerate(frames):
0626         frame_path = sequence_dir / f"roi_{idx:03d}.jpg"
0627         cv2.imwrite(str(frame_path), roi)
0628
0629     return sequence_dir
0630
0631
0632 def prepare_roi_input(roi_input: Path) -> Path:
0633     if not CONFIG.enable_roi_keyframe_preprocess or not Path(roi_input).is_dir():
0634         return roi_input
0635
0636     run_id = f"roi_prepare_{int(time.time())}_{uuid.uuid4().hex[:8]}"
0637     output_root = CONFIG.static_model_dir / "roi_preprocess_outputs" / run_id
0638     keyframe_dir = output_root / "keyframes"
0639     preprocess_dir = output_root / "foreground"
0640
0641     try:
0642         from paper_keyframe_repro import run_keyframe_selection
0643
0644         push_event({"type": "status", "text": "正在从多帧 ROI 中选择最佳帧..."})
0645         best_frame = run_keyframe_selection(
0646             input_dir=roi_input,
0647             output_dir=keyframe_dir,
0648             max_keyframes=min(5, max(1, CONFIG.roi_multiframe_count)),
0649         )
0650         if not best_frame:
0651             raise RuntimeError("keyframe selection returned no best frame")
0652
0653         try:
0654             from paper_preprocess_repro import preprocess_foreground
0655
0656             push_event({"type": "status", "text": "正在进行前景 mask 与 bbox 预处理..."})
0657             report = preprocess_foreground(
0658                 input_path=best_frame,
0659                 output_dir=preprocess_dir,
0660                 white_background=True,
0661             )
0662             clean_path = Path(report.clean_image)
0663             if report.success and clean_path.exists():
0664                 logger.info("ROI 多帧预处理完成: %s", clean_path)
0665                 return clean_path
0666             logger.warning("ROI 前景预处理未成功, 回退到 best_frame: %s", report.error)
0667             return best_frame
0668         except Exception as exc:
0669             logger.exception("ROI 前景预处理异常, 回退到 best_frame: %s", exc)
0670             return best_frame
0671
0672     except Exception as exc:
0673         logger.exception("ROI 多帧关键帧选择异常, 回退到第一帧: %s", exc)
0674         candidates = sorted(Path(roi_input).glob("*.jpg"))
0675         return candidates[0] if candidates else roi_input
0676
0677
0678 # ===== 开源 3D 生成: TripoSR CLI =====
0679 def generate_3d_model_local_triposr(roi_path: Path, keyword: str) -> Tuple[Optional[Path], Optional[str]]:
0680     """
0681     使用本地 TripoSR 生成 GLB。
0682     修复 Windows 下 subprocess 日志 GBK 解码失败的问题。
0683     """
0684     if not CONFIG.triposr_dir:
0685         return None, "未配置 TRIPOS_DIR, 无法调用本地 TripoSR"
0686
0687     triposr_dir = Path(CONFIG.triposr_dir)

```



```

0688 run_py = triposr_dir / "run.py"
0689
0690 if not run_py.exists():
0691     return None, f"未找到 TripoSR run.py: {run_py}"
0692
0693 out_dir = LOCAL_3D_WORK_DIR / f"{safe_filename(keyword)}_{int(time.time())}_{uuid.uuid4().hex[:6]}"
0694 out_dir.mkdir(parents=True, exist_ok=True)
0695
0696 cmd = [
0697     CONFIG.triposr_python,
0698     str(run_py),
0699     str(roi_path.resolve()),
0700     "--output-dir",
0701     str(out_dir.resolve()),
0702     "--model-save-format",
0703     "glb",
0704     "--device",
0705     CONFIG.local_3d_device,
0706 ]
0707
0708 if CONFIG.triposr_bake_texture:
0709     cmd += [
0710         "--bake-texture",
0711         "--texture-resolution",
0712         str(CONFIG.triposr_texture_resolution),
0713     ]
0714
0715 logger.info("调用 TripoSR: %s", " ".join(cmd))
0716 push_event({
0717     "type": "status",
0718     "text": "本地 TripoSR 生成 3D 模型中..."
0719 })
0720
0721 try:
0722     completed = subprocess.run(
0723         cmd,
0724         cwd=str(triposr_dir.resolve()),
0725         capture_output=True,
0726         text=False, # 关键: 不要让 Windows 自动用 GBK 解码
0727         timeout=CONFIG.local_3d_timeout,
0728         check=False,
0729     )
0730
0731     stdout_text = completed.stdout.decode("utf-8", errors="replace") if completed.stdout else ""
0732     stderr_text = completed.stderr.decode("utf-8", errors="replace") if completed.stderr else ""
0733
0734     if completed.returncode != 0:
0735         logger.error("TripoSR returncode: %s", completed.returncode)
0736         logger.error("TripoSR stdout:\n%s", stdout_text[-3000:])
0737         logger.error("TripoSR stderr:\n%s", stderr_text[-3000:])
0738         return None, "TripoSR 生成失败, 请检查终端日志和依赖环境"
0739
0740     glb_candidates = list(out_dir.rglob("*.glb"))
0741
0742     if not glb_candidates:
0743         logger.error("TripoSR 未找到 GLB 输出")
0744         logger.error("TripoSR stdout:\n%s", stdout_text[-3000:])
0745         logger.error("TripoSR stderr:\n%s", stderr_text[-3000:])
0746         return None, "TripoSR 未输出 GLB 文件"
0747
0748     generated_glb = glb_candidates[0]
0749     cache_path = cached_model_path(keyword)
0750
0751     shutil.copy2(generated_glb, cache_path)
0752     record_cache(keyword, cache_path, source="TripoSR")
0753     split_model_for_cache(keyword, cache_path)
0754
0755     logger.info("TripoSR 模型已缓存: %s", cache_path)
0756
0757     return cache_path, None
0758
0759 except subprocess.TimeoutExpired as exc:
0760     stdout_text = ""
0761
0762     if exc.stdout:
0763         if isinstance(exc.stdout, bytes):
0764             stdout_text = exc.stdout.decode("utf-8", errors="replace")
0765         else:
0766             stdout_text = str(exc.stdout)
0767
0768     stderr_text = ""
0769
0770     if exc.stderr:
0771         if isinstance(exc.stderr, bytes):
0772             stderr_text = exc.stderr.decode("utf-8", errors="replace")
0773         else:
0774             stderr_text = str(exc.stderr)
0775
0776     logger.error("TripoSR 生成超时")
0777     logger.error("TripoSR stdout:\n%s", stdout_text[-3000:])

```

```

0778         logger.error("TripoSR stderr:\n%s", stderr_text[-3000:])
0779
0780         return None, "TripoSR 生成超时"
0781
0782     except Exception as exc:
0783         logger.exception("TripoSR 调用异常: %s", exc)
0784         return None, "TripoSR 调用异常"
0785
0786 # ===== 处理 ROI: 开源 VLM + 本地 3D + 缓存 =====
0787
0788 def process_roi_async(roi_path: Path) -> None:
0789     def run() -> None:
0790         try:
0791             push_event({"type": "status", "text": "本地开源模型识别中..."})
0792             roi_path_for_model = prepare_roi_input(roi_path)
0793
0794             result = analyze_image_local_qwen(roi_path_for_model)
0795             if "|" in result:
0796                 explanation, keyword = result.rsplit("|", 1)
0797             else:
0798                 explanation = result
0799                 keyword = "可爱的卡通角色"
0800
0801             explanation = explanation.strip() or "这是一个有趣的物体!"
0802             keyword = normalize_keyword(keyword.strip() or "可爱的卡通角色")
0803
0804             push_event({"type": "knowledge", "text": explanation})
0805             speak_and_push(explanation)
0806
0807         # 缓存命中: 直接加载
0808         cached_path = find_cached_model(keyword)
0809         if cached_path:
0810             if CONFIG.enable_geometry_texture_split and not split_report_path(keyword).exists():
0811                 split_model_for_cache(keyword, cached_path)
0812             logger.info("模型缓存命中: %s -> %s", keyword, cached_path)
0813             push_event({"type": "status", "text": f"缓存命中: {keyword}"})
0814             push_event({
0815                 "type": "model",
0816                 "url": f"/models/model_cache/{cached_path.name}",
0817                 "keyword": keyword,
0818                 "cache_hit": True,
0819                 **progressive_model_meta(keyword, cached_path),
0820             })
0821             return
0822
0823         query_image_for_cache = roi_path_for_model if Path(roi_path_for_model).exists() else roi_path
0824         if CONFIG.enable_policy_cache:
0825             fused_meta = find_fused_cached_model_with_policy_meta(
0826                 keyword=keyword,
0827                 explanation=explanation,
0828                 query_image=query_image_for_cache,
0829             )
0830             if fused_meta:
0831                 policy_decision, policy_info = decide_policy_cache_from_meta(fused_meta)
0832                 policy_path = Path(fused_meta["best_model_path"])
0833                 fused_keyword = str(fused_meta.get("best_keyword") or keyword)
0834                 logger.info(
0835                     "策略缓存判断: enabled=%s, policy_score=%s, decision=%s, text_score=%s, "
0836                     "image_score=%s, weak=%s, strong=%s, best_model_path=%s, fallback_reason=%s",
0837                     policy_info.get("policy_cache_enabled"),
0838                     policy_info.get("policy_score"),
0839                     policy_decision,
0840                     policy_info.get("text_score"),
0841                     policy_info.get("image_score"),
0842                     policy_info.get("weak_threshold"),
0843                     policy_info.get("strong_threshold"),
0844                     policy_info.get("best_model_path"),
0845                     policy_info.get("fallback_reason"),
0846                 )
0847             if policy_decision == "auto_hit" and policy_path.exists():
0848                 if CONFIG.enable_geometry_texture_split and not split_report_path(fused_keyword).exists():
0849                     split_model_for_cache(fused_keyword, policy_path)
0850                 push_event({"type": "status", "text": "策略缓存自动命中, 已复用"})
0851                 push_event({
0852                     "type": "model",
0853                     "url": f"/models/model_cache/{policy_path.name}",
0854                     "keyword": fused_keyword,
0855                     "query_keyword": keyword,
0856                     "cache_hit": True,
0857                     "fused_cache_hit": True,
0858                     "policy_cache_hit": True,
0859                     "policy_score": policy_info.get("policy_score"),
0860                     **progressive_model_meta(fused_keyword, policy_path),
0861                 })
0862                 return
0863             if policy_decision == "auto_hit" and not policy_path.exists():
0864                 logger.warning(
0865                     "策略缓存 auto_hit 但 best_model_path 不存在, 按 miss 回退: %s",
0866                     policy_path,

```

```

0867         )
0868     elif policy_decision == "review":
0869         candidate_model_url = ""
0870         candidate_model_path = ""
0871         if policy_path.exists():
0872             candidate_model_url = f"/models/model_cache/{policy_path.name}"
0873             candidate_model_path = str(policy_path)
0874             review_event = {
0875                 "type": "cache_review",
0876                 "text": "发现相似缓存模型，后续可支持复用",
0877                 "candidate_model_url": candidate_model_url,
0878                 "candidate_model_path": candidate_model_path,
0879                 "policy_score": policy_info.get("policy_score"),
0880                 "text_score": policy_info.get("text_score"),
0881                 "image_score": policy_info.get("image_score"),
0882                 "best_keyword": fused_keyword,
0883                 "best_filename": str(fused_meta.get("best_filename") or policy_path.name),
0884                 "weak_threshold": policy_info.get("weak_threshold"),
0885                 "strong_threshold": policy_info.get("strong_threshold"),
0886                 "review_phase": "hint_only",
0887                 "fallback_reason": "" if policy_path.exists() else "best_model_path_missing",
0888             }
0889             push_event(review_event)
0890             logger.info(
0891                 "策略缓存 review UI 提示已发送: review_ui_event_sent=True, "
0892                 "review_phase=hint_only, policy_decision=review, policy_score=%s, "
0893                 "best_model_path=%s, review_fallback_to_miss=True",
0894                 policy_info.get("policy_score"),
0895                 policy_info.get("best_model_path"),
0896             )
0897             logger.info(
0898                 "策略缓存 review: 当前前端未接入 review, review 按 miss 回退原生成流程。"
0899             )
0900         else:
0901             logger.info("策略缓存 miss, 继续原生成流程。")
0902     else:
0903         fused_hit = find_fused_cached_model(
0904             keyword=keyword,
0905             explanation=explanation,
0906             query_image=query_image_for_cache,
0907             threshold=0.82,
0908         )
0909         if fused_hit:
0910             fused_path, fused_score, fused_keyword = fused_hit
0911             if CONFIG.enable_geometry_texture_split and not split_report_path(fused_keyword).exists():
0912                 split_model_for_cache(fused_keyword, fused_path)
0913             logger.info(
0914                 "融合缓存命中: fused_score=%.4f, model_path=%s",
0915                 fused_score,
0916                 fused_path,
0917             )
0918             push_event({"type": "status", "text": "发现相似缓存模型，已复用"})
0919             push_event({
0920                 "type": "model",
0921                 "url": f"/models/model_cache/{fused_path.name}",
0922                 "keyword": fused_keyword,
0923                 "query_keyword": keyword,
0924                 "cache_hit": True,
0925                 "fused_cache_hit": True,
0926                 "fused_score": round(fused_score, 4),
0927                 **progressive_model_meta(fused_keyword, fused_path),
0928             })
0929             return
0930
0931     # 缓存未命中: 本地 TripoSR 生成
0932     push_event({"type": "status", "text": f"本地生成 3D 模型: {keyword}"})
0933     model_path, error = generate_3d_model_local_triposr(roi_path_for_model, keyword)
0934
0935     if error:
0936         push_event({"type": "error", "text": error})
0937         return
0938
0939     if model_path:
0940         push_event({
0941             "type": "model",
0942             "url": f"/models/model_cache/{model_path.name}",
0943             "keyword": keyword,
0944             "cache_hit": False,
0945             **progressive_model_meta(keyword, model_path),
0946         })
0947         push_event({"type": "status", "text": "本地 3D 模型已生成并缓存"})
0948     else:
0949         push_event({"type": "error", "text": "本地模型生成失败"})
0950
0951     finally:
0952         processing_lock.release()
0953
0954     threading.Thread(target=run, daemon=True).start()
0955

```

```

0956
0957 # ===== 手势识别辅助 =====
0958
0959 class SwipeDetector:
0960     def __init__(self, maxlen: int = 8) -> None:
0961         self.points: Deque[float, float, float] = collections.deque(maxlen=maxlen)
0962         self.last_emit = 0.0
0963
0964     def update(self, x: float, y: float) -> Optional[str]:
0965         now = time.time()
0966         self.points.append((now, x, y))
0967         if len(self.points) < self.points.maxlen:
0968             return None
0969         t0, x0, y0 = self.points[0]
0970         t1, x1, y1 = self.points[-1]
0971         dt = max(0.001, t1 - t0)
0972         dx = x1 - x0
0973         dy = y1 - y0
0974         if dt <= 0.75 and abs(dx) > 0.22 and abs(dy) < 0.14 and now - self.last_emit > 1.0:
0975             self.last_emit = now
0976             self.points.clear()
0977             return "swipe_right" if dx > 0 else "swipe_left"
0978         return None
0979
0980
0981 def finger_extended(hand: List[Any], tip: int, pip: int, margin: float = 0.018) -> bool:
0982     return hand[tip].y < hand[pip].y - margin
0983
0984
0985 def finger_down(hand: List[Any], tip: int, pip: int, margin: float = 0.018) -> bool:
0986     return hand[tip].y > hand[pip].y + margin
0987
0988
0989 def classify_hand_gesture(hand: List[Any]) -> Tuple[str, str, float]:
0990     index_up = finger_extended(hand, 8, 6)
0991     middle_up = finger_extended(hand, 12, 10)
0992     ring_up = finger_extended(hand, 16, 14)
0993     pinky_up = finger_extended(hand, 20, 18)
0994     index_down = finger_down(hand, 8, 6)
0995     middle_down = finger_down(hand, 12, 10)
0996     ring_down = finger_down(hand, 16, 14)
0997     pinky_down = finger_down(hand, 20, 18)
0998     up_count = sum([index_up, middle_up, ring_up, pinky_up])
0999     down_count = sum([index_down, middle_down, ring_down, pinky_down])
1000     if up_count >= 4:
1001         return "open_palm", "张开手掌: 待命", 0.88
1002     if up_count == 0 and down_count >= 3:
1003         return "fist", "握拳: 确认/选中", 0.82
1004     if index_up and not middle_up and not ring_up and not pinky_up:
1005         return "point_up", "食指上指: 向上滚动", 0.86
1006     if index_down and not middle_down and not ring_down and not pinky_down:
1007         return "point_down", "食指下指: 向下滚动", 0.82
1008     if index_up and middle_up and not ring_up and not pinky_up:
1009         return "two_fingers_up", "两指上指: 放大", 0.84
1010     if index_down and middle_down and not ring_down and not pinky_down:
1011         return "two_fingers_down", "两指下指: 缩小", 0.80
1012     return "unknown", "未识别手势", 0.45
1013
1014
1015 def draw_hand(frame: Any, hand: List[Any]) -> None:
1016     h, w, _ = frame.shape
1017     for a, b in HAND_LINES:
1018         p1 = hand[a]
1019         p2 = hand[b]
1020         cv2.line(frame, (int(p1.x * w), int(p1.y * h)), (int(p2.x * w), int(p2.y * h)), (255, 192, 203), 2)
1021     for lm in hand:
1022         cv2.circle(frame, (int(lm.x * w), int(lm.y * h)), 4, (0, 255, 255), -1)
1023
1024
1025 def get_roi_rect_from_points(points: List[Tuple[int, int]], width: int, height: int) -> Optional[Tuple[int, int, int, int]]:
1026     if len(points) < 4:
1027         return None
1028     xs = [p[0] for p in points]
1029     ys = [p[1] for p in points]
1030     xmin = max(0, min(xs))
1031     ymin = max(0, min(ys))
1032     xmax = min(width - 1, max(xs))
1033     ymax = min(height - 1, max(ys))
1034     if xmax <= xmin or ymax <= ymin:
1035         return None
1036     area_ratio = ((xmax - xmin) * (ymax - ymin)) / max(1, width * height)
1037     return (xmin, ymin, xmax, ymax) if area_ratio >= CONFIG.min_roi_area_ratio else None
1038
1039
1040 def is_rect_stable(prev: Optional[Tuple[int, int, int, int]], cur: Tuple[int, int, int, int], width: int, height: int) -> bool:
1041     if prev is None:
1042         return False
1043     px1, py1, px2, py2 = prev
1044     cx1, cy1, cx2, cy2 = cur
1045     prev_center = ((px1 + px2) / 2, (py1 + py2) / 2)

```

```

1046     cur_center = ((cx1 + cx2) / 2, (cy1 + cy2) / 2)
1047     center_shift = dist_px(prev_center, cur_center)
1048     size_shift = abs((px2 - px1) - (cx2 - cx1)) + abs((py2 - py1) - (cy2 - cy1))
1049     return center_shift < min(width, height) * 0.045 and size_shift < min(width, height) * 0.09
1050
1051 # ===== 视频生成器 =====
1052
1053 def generate_video():
1054     if not Path(CONFIG.model_path).exists():
1055         msg = "Hand model not found"
1056         logger.error("手势模型文件不存在: %s", CONFIG.model_path)
1057         push_event({"type": "error", "text": f"手势模型文件不存在: {CONFIG.model_path}"})
1058         jpeg = make_message_jpeg(msg)
1059         while True:
1060             yield b"--frame\r\nContent-Type: image/jpeg\r\n\r\n" + jpeg + b"\r\n"
1061             time.sleep(1)
1062
1063     base_options = tasks.BaseOptions(model_asset_path=CONFIG.model_path)
1064     options = vision.HandLandmarkerOptions(base_options=base_options, num_hands=2, running_mode=vision.RunningMode.IMAGE)
1065     hand_detector = vision.HandLandmarker.create_from_options(options)
1066
1067     cap = cv2.VideoCapture(CONFIG.camera_index)
1068     cap.set(cv2.CAP_PROP_FRAME_WIDTH, CONFIG.camera_width)
1069     cap.set(cv2.CAP_PROP_FRAME_HEIGHT, CONFIG.camera_height)
1070
1071     if not cap.isOpened():
1072         logger.error("摄像头打开失败: index=%s", CONFIG.camera_index)
1073         push_event({"type": "error", "text": "摄像头打开失败, 请检查摄像头权限或 CAMERA_INDEX"})
1074         jpeg = make_message_jpeg("Camera error")
1075         while True:
1076             yield b"--frame\r\nContent-Type: image/jpeg\r\n\r\n" + jpeg + b"\r\n"
1077             time.sleep(1)
1078
1079     stable_frames = 0
1080     cooldown_until = 0.0
1081     prev_dist = None
1082     prev_roi_rect: Optional[Tuple[int, int, int, int]] = None
1083     roi_frame_buffer: Deque[Any] = collections.deque(maxlen=max(1, CONFIG.roi_multiframe_count))
1084     last_gesture = ""
1085     last_gesture_ts = 0.0
1086     swipe = SwipeDetector()
1087
1088     try:
1089         while True:
1090             ret, frame = cap.read()
1091             if not ret:
1092                 logger.warning("读取摄像头帧失败")
1093                 break
1094
1095             frame = cv2.flip(frame, 1)
1096             raw_frame = frame.copy()
1097             h, w, _ = frame.shape
1098             rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
1099             mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)
1100             hand_result = hand_detector.detect(mp_image)
1101
1102             finger_points: List[Tuple[int, int]] = []
1103             active_hands: List[Tuple[float, float]] = []
1104             hands = hand_result.hand_landmarks or []
1105
1106             if hands:
1107                 gesture, detail, confidence = classify_hand_gesture(hands[0])
1108                 swipe_gesture = swipe.update(hands[0][8].x, hands[0][8].y)
1109                 if swipe_gesture:
1110                     gesture = swipe_gesture
1111                     detail = "右滑: 旋转模型" if swipe_gesture == "swipe_right" else "左滑: 旋转模型"
1112                     confidence = 0.90
1113
1114                 now = time.time()
1115                 if gesture != last_gesture or now - last_gesture_ts >= CONFIG.gesture_event_interval:
1116                     push_event({"type": "gesture", "gesture": gesture, "detail": detail, "confidence": confidence})
1117                     last_gesture = gesture
1118                     last_gesture_ts = now
1119
1120                 for hand in hands:
1121                     draw_hand(frame, hand)
1122                     ix = int(hand[8].x * w)
1123                     iy = int(hand[8].y * h)
1124                     tx = int(hand[4].x * w)
1125                     ty = int(hand[4].y * h)
1126                     pinch_distance = dist_norm(hand[8], hand[4])
1127                     is_pinching = pinch_distance < CONFIG.pinch_threshold_norm
1128                     if is_pinching:
1129                         active_hands.append((hand[8].x, hand[8].y))
1130                         cv2.circle(frame, (ix, iy), 12, (0, 255, 0), -1)
1131                         cv2.circle(frame, (tx, ty), 12, (0, 255, 0), -1)
1132                         cv2.line(frame, (ix, iy), (tx, ty), (255, 255, 255), 4)
1133                     else:
1134                         cv2.circle(frame, (ix, iy), 8, (0, 255, 0), -1)

```

```

1136         cv2.circle(frame, (tx, ty), 8, (0, 255, 0), -1)
1137         finger_points.extend([(ix, iy), (tx, ty)])
1138     else:
1139         last_gesture = ""
1140
1141     if len(active_hands) == 1:
1142         push_event({"type": "drag", "active": True, "x": active_hands[0][0], "y": active_hands[0][1]})
1143     elif active_hands:
1144         push_event({"type": "drag", "active": False, "x": 0.5, "y": 0.5})
1145
1146     if len(active_hands) == 2:
1147         (x1, y1), (x2, y2) = active_hands
1148         cur = math.hypot(x2 - x1, y2 - y1)
1149         if prev_dist and prev_dist > 0.001:
1150             factor = max(0.85, min(1.18, cur / prev_dist))
1151             push_event({"type": "scale", "factor": factor})
1152         prev_dist = cur
1153     else:
1154         prev_dist = None
1155
1156     roi_rect = None
1157     if len(hands) >= 2 and len(active_hands) == 0:
1158         roi_rect = get_roi_rect_from_points(finger_points, w, h)
1159
1160     if roi_rect:
1161         if is_rect_stable(prev_roi_rect, roi_rect, w, h):
1162             stable_frames += 1
1163         else:
1164             stable_frames = 1
1165             prev_roi_rect = roi_rect
1166             xmin, ymin, xmax, ymax = roi_rect
1167             roi_frame_buffer.append(raw_frame[ymin:ymax, xmin:xmax].copy())
1168             ready = stable_frames >= CONFIG.trigger_frames
1169             color = (0, 255, 0) if ready else (0, 200, 200)
1170             cv2.rectangle(frame, (xmin, ymin), (xmax, ymax), color, 3 if ready else 2)
1171             cv2.putText(frame, f"Selecting {stable_frames}/{CONFIG.trigger_frames}", (xmin, max(30, ymin - 10)),
1172 cv2.FONT_HERSHEY_SIMPLEX, 0.65, color, 2)
1173         else:
1174             stable_frames = 0
1175             prev_roi_rect = None
1176             roi_frame_buffer.clear()
1177
1178     now = time.time()
1179     if now < cooldown_until:
1180         cv2.putText(frame, "Cooldown...", (10, 120), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
1181
1182     if roi_rect and stable_frames >= CONFIG.trigger_frames and now >= cooldown_until and not processing_lock.locked():
1183         if processing_lock.acquire(blocking=False):
1184             stable_frames = 0
1185             cooldown_until = now + CONFIG.cooldown_seconds
1186             xmin, ymin, xmax, ymax = roi_rect
1187             roi = raw_frame[ymin:ymax, xmin:xmax]
1188             buffered_rois = list(roi_frame_buffer)
1189             if CONFIG.enable_roi_keyframe_preprocess and len(buffered_rois) >= 2:
1190                 if len(buffered_rois) < CONFIG.roi_multiframe_count:
1191                     buffered_rois.append(roi.copy())
1192                 roi_path = save_roi_sequence(buffered_rois)
1193             else:
1194                 roi_path = CONFIG.static_model_dir / f"selected_roi_{int(time.time())}_{uuid.uuid4().hex[:8]}.jpg"
1195                 cv2.imwrite(str(roi_path), roi)
1196             roi_frame_buffer.clear()
1197             logger.info("已保存 ROI 输入: %s", roi_path)
1198             process_roi_async(roi_path)
1199
1200     ok, jpeg = cv2.imencode(".jpg", frame, [cv2.IMWRITE_JPEG_QUALITY, CONFIG.jpeg_quality])
1201     if ok:
1202         yield b"--frame\r\nContent-Type: image/jpeg\r\n\r\n" + jpeg.tobytes() + b"\r\n"
1203     time.sleep(0.01)
1204 finally:
1205     cap.release()
1206     try:
1207         hand_detector.close()
1208     except Exception:
1209         pass
1210     logger.info("视频流资源已释放")
1211
1212 # ===== Flask 路由 =====
1213
1214 @app.route("/")
1215 def index():
1216     return Response(HTML_PAGE, mimetype="text/html; charset=utf-8")
1217
1218
1219 @app.route("/video_feed")
1220 def video_feed():
1221     return Response(generate_video(), mimetype="multipart/x-mixed-replace; boundary=frame")
1222
1223
1224 @app.route("/models/<path:filename>")

```

```

1225 def serve_model(filename: str):
1226     return send_from_directory(str(CONFIG.static_model_dir.resolve()), filename)
1227
1228
1229 @app.route("/events")
1230 def sse():
1231     def stream():
1232         while True:
1233             try:
1234                 msg = sse_queue.get(timeout=0.5)
1235                 yield sse_pack(msg)
1236             except queue.Empty:
1237                 yield ": keep-alive\n\n"
1238
1239     headers = {"Cache-Control": "no-cache", "X-Accel-Buffering": "no", "Connection": "keep-alive"}
1240     return Response(stream(), mimetype="text/event-stream", headers=headers)
1241
1242
1243 @app.route("/health")
1244 def health():
1245     cache_index = load_cache_index()
1246     return {
1247         "ok": True,
1248         "model_exists": Path(CONFIG.model_path).exists(),
1249         "model_path": CONFIG.model_path,
1250         "static_model_dir": str(CONFIG.static_model_dir.resolve()),
1251         "model_cache_dir": str(MODEL_CACHE_DIR.resolve()),
1252         "cache_count": len(cache_index),
1253         "local_vlm_model": CONFIG.local_vlm_model,
1254         "triposr_dir": CONFIG.triposr_dir,
1255         "triposr_configured": bool(CONFIG.triposr_dir and (Path(CONFIG.triposr_dir) / "run.py").exists()),
1256         "geometry_texture_split_enabled": CONFIG.enable_geometry_texture_split,
1257         "progressive_model_loading_enabled": CONFIG.enable_progressive_model_loading,
1258         "roi_keyframe_preprocess_enabled": CONFIG.enable_roi_keyframe_preprocess,
1259         "roi_multiframe_count": CONFIG.roi_multiframe_count,
1260         "model_split_dir": str(MODEL_SPLIT_DIR.resolve()),
1261         "processing": processing_lock.locked(),
1262     }
1263
1264
1265 @app.route("/cache")
1266 def cache_status():
1267     return {"cache_dir": str(MODEL_CACHE_DIR.resolve()), "items": load_cache_index()}
1268
1269
1270 # ===== 程序入口 =====
1271
1272 if __name__ == "__main__":
1273     print("=" * 74)
1274     print("🔗 AR 手势阅读助手 - 开源模型版 v3.6.1")
1275     print("http://localhost:5000")
1276     print("图像理解: Qwen2.5-VL 本地开源模型")
1277     print("3D 生成: TriposR 本地开源 image-to-3D")
1278     print("缓存目录: runtime_assets/model_cache")
1279     print("必填: HAND_MODEL_PATH / TRIPOSR_DIR / TRIPOSR_PYTHON")
1280     print("=" * 74)
1281
1282     if not CONFIG.triposr_dir:
1283         logger.warning("尚未配置 TRIPOSR_DIR。识别可以运行, 但 3D 生成会失败。")
1284     elif not (Path(CONFIG.triposr_dir) / "run.py").exists():
1285         logger.warning("TRIPORSR_DIR 中未找到 run.py: %s", CONFIG.triposr_dir)
1286
1287     app.run(host="0.0.0.0", port=5000, debug=False, threaded=True)
1288

```

缓存策略加载模块: cache_policy_loader.py

文件路径: D:\tool3\py1\pythonProject6\cache_policy_loader.py

代码行数: 87

```

0001 from __future__ import annotations
0002
0003 import argparse
0004 import json
0005 from pathlib import Path
0006 from typing import Any, Dict, Optional
0007
0008
0009 DEFAULT_POLICY_PATH = Path(
0010     "paper_repro_outputs/cache_similarity_eval_v3_real_70/"
0011     "policy_simulation/cache_policy_v3_real_70.json"
0012 )
0013
0014
0015 def load_cache_policy(policy_path: str | Path = DEFAULT_POLICY_PATH) -> Dict[str, Any]:

```



```

0016     path = Path(policy_path)
0017     return json.loads(path.read_text(encoding="utf-8-sig"))
0018
0019
0020 def _to_float(value: Any) -> Optional[float]:
0021     if value is None:
0022         return None
0023     try:
0024         return float(value)
0025     except (TypeError, ValueError):
0026         return None
0027
0028
0029 def compute_policy_score(
0030     text_score: Any,
0031     image_score: Any,
0032     text_weight: float,
0033     image_weight: float,
0034 ) -> float:
0035     text = _to_float(text_score)
0036     image = _to_float(image_score)
0037     if text is None and image is None:
0038         return 0.0
0039     if image is None:
0040         image = text
0041     if text is None:
0042         text = image
0043     return float(text_weight) * float(text) + float(image_weight) * float(image)
0044
0045
0046 def decide_cache_policy(
0047     text_score: Any,
0048     image_score: Any,
0049     policy: Dict[str, Any],
0050 ) -> Dict[str, Any]:
0051     text_weight = float(policy.get("text_weight", 0.5))
0052     image_weight = float(policy.get("image_weight", 0.5))
0053     weak_threshold = float(policy.get("weak_threshold", 0.7))
0054     strong_threshold = float(policy.get("strong_threshold", 0.78))
0055     score = compute_policy_score(text_score, image_score, text_weight, image_weight)
0056
0057     if score >= strong_threshold:
0058         decision = "auto_hit"
0059     elif score >= weak_threshold:
0060         decision = "review"
0061     else:
0062         decision = "miss"
0063
0064     return {
0065         "policy_score": round(score, 6),
0066         "policy_decision": decision,
0067         "weak_threshold": weak_threshold,
0068         "strong_threshold": strong_threshold,
0069         "text_weight": text_weight,
0070         "image_weight": image_weight,
0071     }
0072
0073
0074 def main() -> None:
0075     parser = argparse.ArgumentParser(description="Load cache policy and decide cache reuse action.")
0076     parser.add_argument("--policy-path", default=str(DEFAULT_POLICY_PATH))
0077     parser.add_argument("--text-score", type=float, default=None)
0078     parser.add_argument("--image-score", type=float, default=None)
0079     args = parser.parse_args()
0080
0081     policy = load_cache_policy(args.policy_path)
0082     decision = decide_cache_policy(args.text_score, args.image_score, policy)
0083     print(json.dumps(decision, ensure_ascii=False, indent=2))
0084
0085
0086 if __name__ == "__main__":
0087     main()

```

策略 dry-run 验证模块: cache_policy_dry_run.py

文件路径: D:\tool3\py1\pythonProject6\cache_policy_dry_run.py

代码行数: 191

```

0001 from __future__ import annotations
0002
0003 import argparse
0004 import json
0005 from dataclasses import asdict
0006 from pathlib import Path
0007 from typing import Any, Dict, Optional
0008

```

```

0009 from cache_policy_loader import decide_cache_policy, load_cache_policy
0010 from cache_similarity import build_similarity_index, score_cache_entries
0011
0012
0013 DEFAULT_POLICY_PATH = Path(
0014     "paper_repro_outputs/cache_similarity_eval_v3_real_70/"
0015     "policy_simulation/cache_policy_v3_real_70.json"
0016 )
0017 DEFAULT_OUTPUT_DIR = Path("paper_repro_outputs/cache_similarity_eval_v3_real_70/policy_dry_run")
0018 DEFAULT_CACHE_DIR = Path("runtime_assets/model_cache")
0019
0020
0021 def write_json(path: Path, payload: Dict[str, Any]) -> None:
0022     path.parent.mkdir(parents=True, exist_ok=True)
0023     path.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
0024
0025
0026 def resolve_query_image(query_image: str) -> Optional[Path]:
0027     if not query_image:
0028         return None
0029     path = Path(query_image)
0030     if not path.exists():
0031         raise FileNotFoundError(f"query_image does not exist: {path}")
0032     return path
0033
0034
0035 def build_entries(cache_dir: Path, output_dir: Path):
0036     index_path = output_dir / "cache_similarity_index_dry_run.json"
0037     return build_similarity_index(
0038         cache_dir=cache_dir,
0039         reference_dir=cache_dir / "reference_images",
0040         output_path=index_path,
0041     )
0042
0043
0044 def decision_explanation(policy_decision: str) -> str:
0045     if policy_decision == "auto_hit":
0046         return "当前样本可自动复用缓存模型，但本脚本不实际加载模型。"
0047     if policy_decision == "review":
0048         return "当前样本进入 review 区，未来前端应提示用户确认是否复用。"
0049     return "当前样本不复用缓存，未来应回退到原生成流程。"
0050
0051
0052 def run_dry_run(
0053     query_text: str,
0054     query_image: Optional[Path],
0055     cache_dir: Path,
0056     policy_path: Path,
0057     output_dir: Path,
0058 ) -> Dict[str, Any]:
0059     output_dir.mkdir(parents=True, exist_ok=True)
0060     policy = load_cache_policy(policy_path)
0061     fallback_reason = ""
0062     best = None
0063     entries = []
0064     results = []
0065
0066     try:
0067         entries = build_entries(cache_dir, output_dir)
0068         if not entries:
0069             fallback_reason = "no_cache_entries"
0070         else:
0071             results = score_cache_entries(
0072                 entries,
0073                 query_text=query_text,
0074                 query_image=query_image,
0075                 text_weight=float(policy.get("text_weight", 0.5)),
0076                 image_weight=float(policy.get("image_weight", 0.5)),
0077                 threshold=float(policy.get("weak_threshold", 0.7)),
0078             )
0079             best = results[0] if results else None
0080             if best is None:
0081                 fallback_reason = "no_similarity_result"
0082     except Exception as exc:
0083         fallback_reason = f"similarity_failed: {exc}"
0084
0085     text_score = best.text_score if best else None
0086     image_score = best.image_score if best else None
0087     decision = decide_cache_policy(text_score, image_score, policy)
0088     best_model_path = best.model_path if best else ""
0089
0090     if best_model_path and not Path(best_model_path).exists():
0091         fallback_reason = "best_model_path_missing"
0092
0093     result = {
0094         "query_text": query_text,
0095         "query_image": str(query_image) if query_image else "",
0096         "text_score": text_score,
0097         "image_score": image_score,
0098         "policy_score": decision["policy_score"],

```

```

0099         "policy_decision": decision["policy_decision"],
0100         "weak_threshold": decision["weak_threshold"],
0101         "strong_threshold": decision["strong_threshold"],
0102         "best_keyword": best.keyword if best else "",
0103         "best_filename": best.filename if best else "",
0104         "best_model_path": best_model_path,
0105         "fallback_reason": fallback_reason,
0106         "cache_dir": str(cache_dir),
0107         "policy_path": str(policy_path),
0108         "cache_entry_count": len(entries),
0109     }
0110
0111     write_json(output_dir / "dry_run_result.json", result)
0112     write_report(output_dir / "dry_run_report.md", result)
0113
0114     if results:
0115         write_json(
0116             output_dir / "dry_run_all_candidates.json",
0117             {"results": [asdict(item) for item in results]},
0118         )
0119
0120     return result
0121
0122
0123 def write_report(path: Path, result: Dict[str, Any]) -> None:
0124     decision = result["policy_decision"]
0125     text = f"""# 缓存策略 dry-run 测试报告
0126
0127 ## 输入
0128
0129 - query_text: {result.get('query_text') or 'N/A'}
0130 - query_image: {result.get('query_image') or 'N/A'}
0131 - cache_dir: {result.get('cache_dir')}
0132 - policy_path: {result.get('policy_path')}
0133
0134 ## 最佳缓存候选
0135
0136 - best_keyword: {result.get('best_keyword') or 'N/A'}
0137 - best_filename: {result.get('best_filename') or 'N/A'}
0138 - best_model_path: {result.get('best_model_path') or 'N/A'}
0139 - best_model_path 是否存在: {bool(result.get('best_model_path') and Path(str(result.get('best_model_path'))).exists())}
0140
0141 ## 分数与决策
0142
0143 - text_score: {result.get('text_score')}
0144 - image_score: {result.get('image_score')}
0145 - policy_score: {result.get('policy_score')}
0146 - policy_decision: {decision}
0147 - 是否可 auto_hit: {decision == 'auto_hit'}
0148 - 是否需要 review: {decision == 'review'}
0149 - 是否 miss: {decision == 'miss'}
0150 - fallback_reason: {result.get('fallback_reason') or 'N/A'}
0151
0152 ## 决策解释
0153
0154 {decision_explanation(decision)}
0155
0156 ## 注意
0157
0158 本脚本只进行 dry-run 判断，不加载模型、不生成模型、不修改 `plus.py`，也不接入前端。
0159 """
0160     path.write_text(text, encoding="utf-8")
0161
0162
0163 def main() -> None:
0164     parser = argparse.ArgumentParser(description="Dry-run cache policy decision without touching plus.py.")
0165     parser.add_argument("--query-text", default="")
0166     parser.add_argument("--query-image", default="")
0167     parser.add_argument("--cache-dir", default=str(DEFAULT_CACHE_DIR))
0168     parser.add_argument("--policy-path", default=str(DEFAULT_POLICY_PATH))
0169     parser.add_argument("--output-dir", default=str(DEFAULT_OUTPUT_DIR))
0170     args = parser.parse_args()
0171
0172     query_image = resolve_query_image(args.query_image)
0173     result = run_dry_run(
0174         query_text=args.query_text,
0175         query_image=query_image,
0176         cache_dir=Path(args.cache_dir),
0177         policy_path=Path(args.policy_path),
0178         output_dir=Path(args.output_dir),
0179     )
0180
0181     print("=" * 72)
0182     print(f"dry_run_result.json: {Path(args.output_dir) / 'dry_run_result.json'}")
0183     print(f"dry_run_report.md: {Path(args.output_dir) / 'dry_run_report.md'}")
0184     print(f"policy_decision: {result['policy_decision']}")
0185     print(f"policy_score: {result['policy_score']}")
0186     print(f"best_model_path: {result['best_model_path']}")
0187     print("=" * 72)

```

```
0188
0189
0190 if __name__ == "__main__":
0191     main()
```

相似度方法对比脚本: make_similarity_method_comparison.py

文件路径: D:\tool3\py1\pythonProject6\make_similarity_method_comparison.py

代码行数: 310

```
0001 from __future__ import annotations
0002
0003 import argparse
0004 import csv
0005 import json
0006 from pathlib import Path
0007 from typing import Any, Dict, List, Optional
0008
0009
0010 DEFAULT_EVAL_DIR = Path("paper_repro_outputs/cache_similarity_eval_v3_real_70")
0011 DEFAULT_OUTPUT_DIR = DEFAULT_EVAL_DIR / "similarity_method_comparison"
0012
0013
0014 def read_csv_rows(path: Path) -> List[Dict[str, Any]]:
0015     with path.open("r", encoding="utf-8-sig", newline="") as f:
0016         return list(csv.DictReader(f))
0017
0018
0019 def read_json(path: Path) -> Dict[str, Any]:
0020     return json.loads(path.read_text(encoding="utf-8-sig"))
0021
0022
0023 def to_float(value: Any, default: float = 0.0) -> float:
0024     try:
0025         if value is None or value == "":
0026             return default
0027         return float(value)
0028     except (TypeError, ValueError):
0029         return default
0030
0031
0032 def to_bool(value: Any) -> bool:
0033     return str(value).strip().lower() in {"1", "true", "yes", "y"}
0034
0035
0036 def find_weight_row(rows: List[Dict[str, Any]], text_weight: float) -> Optional[Dict[str, Any]]:
0037     for row in rows:
0038         if abs(to_float(row.get("text_weight")) - text_weight) < 1e-9:
0039             return row
0040     return None
0041
0042
0043 def compute_single_threshold_counts(
0044     summary_rows: List[Dict[str, Any]],
0045     text_weight: float,
0046     image_weight: float,
0047     threshold: float,
0048 ) -> Dict[str, int]:
0049     auto_hit_count = 0
0050     miss_count = 0
0051     for row in summary_rows:
0052         text = to_float(row.get("text_score"))
0053         image = to_float(row.get("image_score"), text)
0054         score = text_weight * text + image_weight * image
0055         if score >= threshold:
0056             auto_hit_count += 1
0057         else:
0058             miss_count += 1
0059     return {"auto_hit_count": auto_hit_count, "review_count": 0, "miss_count": miss_count}
0060
0061
0062 def build_method_row(
0063     method: str,
0064     weight_row: Dict[str, Any],
0065     counts: Dict[str, int],
0066     threshold_label: str,
0067 ) -> Dict[str, Any]:
0068     return {
0069         "method": method,
0070         "text_weight": to_float(weight_row.get("text_weight")),
0071         "image_weight": to_float(weight_row.get("image_weight")),
0072         "threshold": threshold_label,
0073         "accuracy": to_float(weight_row.get("accuracy")),
0074         "precision": to_float(weight_row.get("precision")),
0075         "recall": to_float(weight_row.get("recall")),
0076         "false_hit_rate": to_float(weight_row.get("false_hit_rate")),
```

```

0077         "false_miss_rate": to_float(weight_row.get("false_miss_rate")),
0078         "near_positive_false_miss_rate": to_float(
0079             weight_row.get("near_positive_false_miss_rate")
0080         ),
0081         "hard_negative_false_hit_rate": to_float(
0082             weight_row.get("hard_negative_false_hit_rate")
0083         ),
0084         "auto_hit_count": counts["auto_hit_count"],
0085         "review_count": counts["review_count"],
0086         "miss_count": counts["miss_count"],
0087     }
0088
0089
0090 def has_garbled(text: str) -> bool:
0091     return "???" in text or "\ufffd" in text
0092
0093
0094 def main() -> None:
0095     parser = argparse.ArgumentParser(description="Compare text-only, image-only, fusion, and dual-threshold methods.")
0096     parser.add_argument("--eval-dir", default=str(DEFAULT_EVAL_DIR))
0097     parser.add_argument("--output-dir", default=str(DEFAULT_OUTPUT_DIR))
0098     args = parser.parse_args()
0099
0100     eval_dir = Path(args.eval_dir)
0101     output_dir = Path(args.output_dir)
0102     output_dir.mkdir(parents=True, exist_ok=True)
0103
0104     summary_csv = eval_dir / "summary.csv"
0105     fusion_best_csv = eval_dir / "fusion_weight_ablation" / "fusion_weight_best_summary.csv"
0106     review_json = eval_dir / "borderline_threshold_analysis" / "recommended_review_threshold.json"
0107
0108     summary_rows = read_csv_rows(summary_csv)
0109     fusion_best_rows = read_csv_rows(fusion_best_csv)
0110     review_data = read_json(review_json)
0111
0112     text_only = find_weight_row(fusion_best_rows, 1.0)
0113     image_only = find_weight_row(fusion_best_rows, 0.0)
0114     rule_fusion = find_weight_row(fusion_best_rows, 0.5)
0115
0116     if text_only is None or image_only is None or rule_fusion is None:
0117         missing = []
0118         if text_only is None:
0119             missing.append("text-only")
0120         if image_only is None:
0121             missing.append("image-only")
0122         if rule_fusion is None:
0123             missing.append("rule-fusion")
0124         raise SystemExit(f"Missing required method rows: {' '.join(missing)}")
0125
0126     text_threshold = to_float(text_only.get("best_threshold"))
0127     image_threshold = to_float(image_only.get("best_threshold"))
0128     fusion_threshold = to_float(rule_fusion.get("best_threshold"))
0129
0130     comparison_rows: List[Dict[str, Any]] = []
0131     comparison_rows.append(
0132         build_method_row(
0133             "text-only",
0134             text_only,
0135             compute_single_threshold_counts(summary_rows, 1.0, 0.0, text_threshold),
0136             str(text_threshold),
0137         )
0138     )
0139     comparison_rows.append(
0140         build_method_row(
0141             "image-only",
0142             image_only,
0143             compute_single_threshold_counts(summary_rows, 0.0, 1.0, image_threshold),
0144             str(image_threshold),
0145         )
0146     )
0147     comparison_rows.append(
0148         build_method_row(
0149             "rule-fusion",
0150             rule_fusion,
0151             compute_single_threshold_counts(summary_rows, 0.5, 0.5, fusion_threshold),
0152             str(fusion_threshold),
0153         )
0154     )
0155
0156     dual_metrics = review_data.get("key_metrics", review_data)
0157     dual_row = {
0158         "method": "dual-threshold fusion",
0159         "text_weight": 0.5,
0160         "image_weight": 0.5,
0161         "threshold": f"{review_data.get('recommended_weak_threshold', 0.7)} / {review_data.get('recommended_strong_threshold', 0.78)}",
0162         "accuracy": "",
0163         "precision": "",
0164         "recall": to_float(dual_metrics.get("recall_if_review_accepted")),
0165         "false_hit_rate": to_float(dual_metrics.get("auto_false_hit_rate")),
0166         "false_miss_rate": to_float(dual_metrics.get("false_miss_rate")),
0167         "near_positive_false_miss_rate": (

```

```

0168         round(
0169             to_float(dual_metrics.get("near_positive_miss_count"))
0170             / max(
0171                 1,
0172                 to_float(dual_metrics.get("near_positive_auto_hit_count"))
0173                 + to_float(dual_metrics.get("near_positive_review_count"))
0174                 + to_float(dual_metrics.get("near_positive_miss_count")),
0175             ),
0176             4,
0177         )
0178     ),
0179     "hard_negative_false_hit_rate": 0.0
0180     if to_float(dual_metrics.get("hard_negative_auto_false_hit_count")) == 0
0181     else "",
0182     "auto_hit_count": int(to_float(dual_metrics.get("auto_hit_count"))),
0183     "review_count": int(to_float(dual_metrics.get("review_count"))),
0184     "miss_count": int(to_float(dual_metrics.get("miss_count"))),
0185 }
0186 comparison_rows.append(dual_row)
0187
0188 csv_path = output_dir / "similarity_method_comparison.csv"
0189 json_path = output_dir / "similarity_method_comparison.json"
0190 report_path = output_dir / "similarity_method_comparison_report.md"
0191
0192 fieldnames = [
0193     "method",
0194     "text_weight",
0195     "image_weight",
0196     "threshold",
0197     "accuracy",
0198     "precision",
0199     "recall",
0200     "false_hit_rate",
0201     "false_miss_rate",
0202     "near_positive_false_miss_rate",
0203     "hard_negative_false_hit_rate",
0204     "auto_hit_count",
0205     "review_count",
0206     "miss_count",
0207 ]
0208 with csv_path.open("w", encoding="utf-8-sig", newline="") as f:
0209     writer = csv.DictWriter(f, fieldnames=fieldnames)
0210     writer.writeheader()
0211     writer.writerows(comparison_rows)
0212
0213 payload = {
0214     "found_text_only": text_only is not None,
0215     "found_image_only": image_only is not None,
0216     "found_rule_fusion": rule_fusion is not None,
0217     "found_dual_threshold_fusion": review_json.exists(),
0218     "recommended_method": "dual-threshold fusion",
0219     "methods": comparison_rows,
0220     "source_files": {
0221         "summary_csv": str(summary_csv),
0222         "fusion_weight_best_summary_csv": str(fusion_best_csv),
0223         "recommended_review_threshold_json": str(review_json),
0224     },
0225 }
0226 json_path.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
0227
0228 table = "\n".join(
0229     [
0230         "| method | text_weight | image_weight | threshold | accuracy | precision | recall | false_hit_rate | false_miss_rate |"
0231         "near_positive_false_miss_rate | hard_negative_false_hit_rate | auto_hit_count | review_count | miss_count |",
0232         *["|---|---|---|---|---|---|---|---|---|---|---|---|---|",
0233            " | {method} | {text_weight} | {image_weight} | {threshold} | {accuracy} | {precision} | {recall} | {false_hit_rate} |"
0234            "{false_miss_rate} | {near_positive_false_miss_rate} | {hard_negative_false_hit_rate} | {auto_hit_count} | {review_count} | {miss_count}"
0235            "|".format(**row)
0236            for row in comparison_rows
0237         ],
0238     )
0239 )
0240
0241 report = f"""## 纯文本 / 纯图像 / 图文融合相似度方法对比报告
0242
0243 ## 1. 对比目的
0244
0245 该报告对应导师会议中“验证不同相似度判断方法”的要求。会议中明确指出，缓存复用能否成为研究问题，关键在于相似度如何定义，以及不同相似度判断方法是否会影响
0246 缓存命中率、误命中率和生成效率。
0247
0248 ## 2. 对比方法
0249
0250 - text-only: 仅使用文本相似度, text_weight=1.0, image_weight=0.0。
0251 - image-only: 仅使用图像相似度, text_weight=0.0, image_weight=1.0。
0252 - rule fusion: 规则融合, score = 0.5 * text_score + 0.5 * image_score。
0253 - dual-threshold fusion: 双阈值融合, 0.7 <= score < 0.78 进入 review, score >= 0.78 自动复用。

```

```

0254 ## 3. 指标对比表
0255
0256 {table}
0257
0258 ## 4. 结果分析
0259
0260 纯文本方法的优点是可解释性强，能直接利用 VLM 输出的语义标签；缺点是容易受到描述随机性影响。同一目标在不同轮次中可能被描述成粗粒度或细粒度不同文本，导致相似度不稳定。
0261
0262 纯图像方法的优点是能绕开文本描述不一致的问题，直接利用视觉特征；缺点是容易受到光照、遮挡、裁剪、伪影和前景 mask 质量影响。在当前 v3_real_70 结果中，image-only 的 false_hit_rate 高于规则融合，说明纯图像判断更容易引入误复用风险。
0263
0264 图文融合方法在当前样本上表现更均衡。0.5 / 0.5 的规则融合保持 false_hit_rate = 0，同时 recall = 0.6111，说明文本和图像互补后能够兼顾安全性和复用能力。
0265
0266 双阈值策略的意义在于把一次性 hit / miss 改成 auto_hit / review / miss 三支。高分样本自动复用，边界样本进入用户确认区，低分样本回退生成。当前 0.7 / 0.78 策略让 10 个 near_positive 边界样本进入 review，且 review_false_candidate_count = 0。
0267
0268 当前先推荐规则融合 baseline，而不是马上训练 MLP，原因是样本量只有 70，仍偏少。规则融合和双阈值可解释、可控、可回退，更适合当前阶段形成稳定实验链路。MLP 可以作为样本扩展到 80~100 后的后续工作。
0269
0270 ## 5. 当前推荐策略
0271
0272 score = 0.5 * text_score + 0.5 * image_score
0273
0274 score >= 0.78: auto_hit
0275 0.7 <= score < 0.78: review
0276 score < 0.7: miss
0277
0278 推荐理由：
0279
0280 - false_hit_rate 保持较低；
0281 - auto_hit 可跳过 TripoSR；
0282 - review 区可保守处理；
0283 - 规则可解释；
0284 - 当前样本量只有 70，暂不适合直接训练 MLP。
0285
0286 ## 6. 与多用户缓存仿真的关系
0287
0288 多用户缓存仿真的基础仍然依赖可靠的相似度判断。只有相似度判断足够稳定，多用户缓存共享才不会出现错误复用。因此，相似度方法对比是多用户协同缓存实验的前置基础。
0289
0290 如果后续要做 user_A / user_B 缓存共享实验，应先使用当前推荐的图文融合双阈值策略来判断远端缓存是否可复用，再比较远端传输耗时和本地重新生成耗时。
0291 """
0292     report_path.write_text(report, encoding="utf-8")
0293
0294     combined = report + json_path.read_text(encoding="utf-8")
0295     output = {
0296         "found_text_only": text_only is not None,
0297         "found_image_only": image_only is not None,
0298         "found_rule_fusion": rule_fusion is not None,
0299         "found_dual_threshold_fusion": review_json.exists(),
0300         "recommended_method": "dual-threshold fusion",
0301         "garbled_detected": has_garbled(combined),
0302         "report_path": str(report_path),
0303         "csv_path": str(csv_path),
0304         "json_path": str(json_path),
0305     }
0306     print(json.dumps(output, ensure_ascii=False, indent=2))
0307
0308
0309 if __name__ == "__main__":
0310     main()

```

多用户缓存仿真脚本：simulate_multi_user_cache_reuse.py

文件路径：D:\tool3\py1\pythonProject6\simulate_multi_user_cache_reuse.py

代码行数：256

```

0001 from __future__ import annotations
0002
0003 import argparse
0004 import csv
0005 import json
0006 from pathlib import Path
0007 from statistics import mean
0008 from typing import Any, Dict, Iterable, List
0009
0010 from cache_policy_loader import decide_cache_policy, load_cache_policy
0011
0012
0013 DEFAULT_SUMMARY_CSV = Path("paper_repro_outputs/cache_similarity_eval_v3_real_70/summary.csv")
0014 DEFAULT_POLICY_PATH = Path("runtime_assets/cache_policy.json")

```



```

0015 DEFAULT_OUTPUT_DIR = Path(
0016     "paper_repro_outputs/cache_similarity_eval_v3_real_70/multi_user_cache_simulation"
0017 )
0018
0019
0020 def parse_bool(value: Any) -> bool:
0021     return str(value).strip().lower() in {"1", "true", "yes", "y"}
0022
0023
0024 def parse_float(value: Any, default: float = 0.0) -> float:
0025     try:
0026         if value is None or value == "":
0027             return default
0028         return float(value)
0029     except (TypeError, ValueError):
0030         return default
0031
0032
0033 def load_rows(path: Path) -> List[Dict[str, Any]]:
0034     with path.open("r", encoding="utf-8-sig", newline="") as f:
0035         return list(csv.DictReader(f))
0036
0037
0038 def write_csv(path: Path, rows: Iterable[Dict[str, Any]], fieldnames: List[str]) -> None:
0039     with path.open("w", encoding="utf-8-sig", newline="") as f:
0040         writer = csv.DictWriter(f, fieldnames=fieldnames)
0041         writer.writeheader()
0042         writer.writerows(rows)
0043
0044
0045 def simulate(args: argparse.Namespace) -> Dict[str, Any]:
0046     summary_csv = Path(args.summary_csv)
0047     output_dir = Path(args.output_dir)
0048     output_dir.mkdir(parents=True, exist_ok=True)
0049
0050     rows = load_rows(summary_csv)
0051     policy = load_cache_policy(args.policy_path)
0052
0053     result_rows: List[Dict[str, Any]] = []
0054     for idx, row in enumerate(rows, start=1):
0055         text_score = parse_float(row.get("text_score"))
0056         image_score = parse_float(row.get("image_score"), text_score)
0057         elapsed_ms = parse_float(row.get("elapsed_ms"))
0058         should_hit = parse_bool(row.get("should_hit"))
0059         decision = decide_cache_policy(text_score, image_score, policy)
0060         policy_decision = decision["policy_decision"]
0061         best_model_path = str(row.get("best_model_path") or "")
0062         model_exists = Path(best_model_path).exists()
0063
0064         baseline_latency_ms = args.generation_ms
0065         auto_reuse = policy_decision == "auto_hit" and model_exists
0066         review_candidate = policy_decision == "review" and model_exists
0067         miss = policy_decision == "miss" or (
0068             policy_decision in {"auto_hit", "review"} and not model_exists
0069         )
0070
0071         if auto_reuse:
0072             simulated_latency_ms = elapsed_ms + args.network_delay_ms + args.model_load_ms
0073             tripopr_called = False
0074             fallback_to_generation = False
0075             effective_action = "remote_cache_reuse"
0076         elif review_candidate:
0077             reuse_latency = elapsed_ms + args.network_delay_ms + args.model_load_ms
0078             fallback_latency = elapsed_ms + args.generation_ms
0079             simulated_latency_ms = (
0080                 args.review_accept_rate * reuse_latency
0081                 + (1.0 - args.review_accept_rate) * fallback_latency
0082             )
0083             tripopr_called = args.review_accept_rate < 1.0
0084             fallback_to_generation = args.review_accept_rate < 1.0
0085             effective_action = "review_expected_reuse"
0086         else:
0087             simulated_latency_ms = elapsed_ms + args.generation_ms
0088             tripopr_called = False
0089             fallback_to_generation = True
0090             effective_action = "fallback_generation_not_executed"
0091
0092     saved_latency_ms = baseline_latency_ms - simulated_latency_ms
0093     result_rows.append(
0094         {
0095             "index": idx,
0096             "image": row.get("image", ""),
0097             "sample_type": row.get("sample_type", ""),
0098             "should_hit": should_hit,
0099             "text_score": text_score,
0100             "image_score": image_score,
0101             "policy_score": decision["policy_score"],
0102             "policy_decision": policy_decision,
0103             "best_keyword": row.get("best_keyword", ""),
0104             "best_model_path": best_model_path,
0105             "best_model_path_exists": model_exists,

```

```

0106         "auto_reuse": auto_reuse,
0107         "review_candidate": review_candidate,
0108         "fallback_to_generation": fallback_to_generation,
0109         "triposr_called_by_simulation": triposr_called,
0110         "effective_action": effective_action,
0111         "baseline_latency_ms": round(baseline_latency_ms, 3),
0112         "simulated_latency_ms": round(simulated_latency_ms, 3),
0113         "saved_latency_ms": round(saved_latency_ms, 3),
0114         "auto_false_hit": auto_reuse and not should_hit,
0115         "review_false_candidate": review_candidate and not should_hit,
0116         "false_miss": miss and should_hit,
0117     }
0118 )
0119
0120 total = len(result_rows)
0121 auto_hit_rows = [r for r in result_rows if r["policy_decision"] == "auto_hit"]
0122 review_rows = [r for r in result_rows if r["policy_decision"] == "review"]
0123 miss_rows = [r for r in result_rows if r["policy_decision"] == "miss"]
0124 should_hit_rows = [r for r in result_rows if r["should_hit"]]
0125 should_not_hit_rows = [r for r in result_rows if not r["should_hit"]]
0126 auto_false_hit_rows = [r for r in result_rows if r["auto_false_hit"]]
0127 review_false_rows = [r for r in result_rows if r["review_false_candidate"]]
0128 false_miss_rows = [r for r in result_rows if r["false_miss"]]
0129 true_reuse_rows = [r for r in result_rows if r["should_hit"] and (r["auto_reuse"] or r["review_candidate"])]
0130
0131 baseline_total = sum(float(r["baseline_latency_ms"]) for r in result_rows)
0132 simulated_total = sum(float(r["simulated_latency_ms"]) for r in result_rows)
0133 saved_total = baseline_total - simulated_total
0134
0135 summary = {
0136     "summary_csv": str(summary_csv),
0137     "policy_path": str(args.policy_path),
0138     "total_samples": total,
0139     "generation_ms": args.generation_ms,
0140     "network_delay_ms": args.network_delay_ms,
0141     "model_load_ms": args.model_load_ms,
0142     "review_accept_rate": args.review_accept_rate,
0143     "baseline_total_latency_ms": round(baseline_total, 3),
0144     "multi_user_simulated_total_latency_ms": round(simulated_total, 3),
0145     "saved_latency_ms": round(saved_total, 3),
0146     "saved_latency_seconds": round(saved_total / 1000.0, 3),
0147     "speedup_ratio": round(baseline_total / simulated_total, 4) if simulated_total else None,
0148     "avg_saved_latency_per_sample_ms": round(saved_total / total, 3) if total else 0,
0149     "should_hit_count": len(should_hit_rows),
0150     "should_not_hit_count": len(should_not_hit_rows),
0151     "auto_hit_count": len(auto_hit_rows),
0152     "review_count": len(review_rows),
0153     "miss_count": len(miss_rows),
0154     "auto_false_hit_count": len(auto_false_hit_rows),
0155     "review_false_candidate_count": len(review_false_rows),
0156     "false_miss_count": len(false_miss_rows),
0157     "auto_false_hit_rate": round(
0158         len(auto_false_hit_rows) / len(should_not_hit_rows), 6
0159     )
0160     if should_not_hit_rows
0161     else 0,
0162     "recall_if_review_accepted": round(len(true_reuse_rows) / len(should_hit_rows), 6)
0163     if should_hit_rows
0164     else 0,
0165     "avg_similarity_elapsed_ms": round(
0166         mean(parse_float(r.get("elapsed_ms")) for r in rows), 3
0167     )
0168     if rows
0169     else 0,
0170     "notes": (
0171         "This is an offline multi-user cache reuse simulation. It does not call Qwen, "
0172         "does not call TriposR, and does not transfer or load GLB files."
0173     ),
0174 }
0175
0176 fieldnames = list(result_rows[0].keys()) if result_rows else []
0177 detail_csv = output_dir / "multi_user_cache_simulation_results.csv"
0178 summary_json = output_dir / "multi_user_cache_simulation_summary.json"
0179 report_md = output_dir / "multi_user_cache_simulation_report.md"
0180 write_csv(detail_csv, result_rows, fieldnames)
0181 summary_json.write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
0182
0183 report = f"""# 多用户缓存复用仿真实验报告
0184
0185 ## 1. 实验目的
0186
0187 本实验用于回应会议中提出的“多用户 AR 缓存共享”方向：当一个用户已经生成过 3D 模型时，另一个用户是否可以通过图文相似度判断复用该缓存模型，从而用传输延迟换取生成算力，减少重复 TriposR 调用。
0188
0189 ## 2. 仿真设置
0190
0191 - 输入结果: `{summary_csv}`
0192 - 策略配置: `{args.policy_path}`
0193 - generation_ms = {args.generation_ms}
0194 - network_delay_ms = {args.network_delay_ms}

```

```

0195 - model_load_ms = {args.model_load_ms}
0196 - review_accept_rate = {args.review_accept_rate}
0197
0198 本轮为离线仿真，不调用 Qwen，不调用 TripoSR，不真实传输 GLB。
0199
0200 ## 3. 核心结果
0201
0202 - total_samples = {summary['total_samples']}
0203 - auto_hit_count = {summary['auto_hit_count']}
0204 - review_count = {summary['review_count']}
0205 - miss_count = {summary['miss_count']}
0206 - auto_false_hit_count = {summary['auto_false_hit_count']}
0207 - review_false_candidate_count = {summary['review_false_candidate_count']}
0208 - recall_if_review_accepted = {summary['recall_if_review_accepted']}
0209 - baseline_total_latency_ms = {summary['baseline_total_latency_ms']}
0210 - multi_user_simulated_total_latency_ms = {summary['multi_user_simulated_total_latency_ms']}
0211 - saved_latency_seconds = {summary['saved_latency_seconds']}
0212 - speedup_ratio = {summary['speedup_ratio']}
0213 - avg_saved_latency_per_sample_ms = {summary['avg_saved_latency_per_sample_ms']}
0214
0215 ## 4. 阶段结论
0216
0217 在当前参数下，多用户缓存复用仿真显示：如果远端缓存命中能够通过图文相似度策略筛出，并且传输延迟低于重新生成耗时，则该机制具备降低总等待时间的潜力。
0218
0219 该实验目前只验证“时间账”和“误复用风险统计”，下一步可以把它扩展为单机双进程或局域网双设备验证。
0220
0221 ## 5. 下一步
0222
0223 1. 单机启动两个 AR 实例，模拟 user_A / user_B；
0224 2. 固定网络传输延迟，例如 20 秒；
0225 3. 记录远端缓存命中时是否跳过 TripoSR；
0226 4. 统计生成耗时、传输耗时、总耗时和误复用；
0227 5. 若稳定，再考虑真实局域网双设备实验。
0228
0229 """
0230     report_md.write_text(report, encoding="utf-8")
0231
0232     summary.update(
0233         {
0234             "detail_csv": str(detail_csv),
0235             "summary_json": str(summary_json),
0236             "report_md": str(report_md),
0237         }
0238     )
0239     return summary
0240
0241 def main() -> None:
0242     parser = argparse.ArgumentParser(description="Offline multi-user cache reuse simulation.")
0243     parser.add_argument("--summary-csv", default=str(DEFAULT_SUMMARY_CSV))
0244     parser.add_argument("--policy-path", default=str(DEFAULT_POLICY_PATH))
0245     parser.add_argument("--output-dir", default=str(DEFAULT_OUTPUT_DIR))
0246     parser.add_argument("--generation-ms", type=float, default=51236)
0247     parser.add_argument("--network-delay-ms", type=float, default=20000)
0248     parser.add_argument("--model-load-ms", type=float, default=1000)
0249     parser.add_argument("--review-accept-rate", type=float, default=1.0)
0250     args = parser.parse_args()
0251     result = simulate(args)
0252     print(json.dumps(result, ensure_ascii=False, indent=2))
0253
0254
0255 if __name__ == "__main__":
0256     main()

```

v3_real 实验运行脚本：run_cache_v3_experiment.py

文件路径：D:\tool3\py1\pythonProject6\run_cache_v3_experiment.py

代码行数：241

```

0001 from __future__ import annotations
0002
0003 import argparse
0004 import json
0005 import subprocess
0006 import sys
0007 from pathlib import Path
0008 from typing import Any, Dict, List, Tuple
0009
0010
0011 IMAGE_EXTS = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}
0012 SAMPLE_TYPES = ["positive", "near_positive", "hard_negative", "negative"]
0013 DEFAULT_THRESHOLDS = "0.5,0.6,0.7,0.75,0.8,0.82,0.85,0.9"
0014 DEFAULT_SOURCE_ROOT = Path("cache_test_v3_real")
0015 DEFAULT_DATASET_DIR = Path("paper_repro_outputs/cache_similarity_dataset_v3_real")
0016 DEFAULT_OUTPUT_DIR = Path("paper_repro_outputs/cache_similarity_eval_v3_real")

```

```

0017
0018
0019 def read_json(path: Path) -> Any:
0020     return json.loads(path.read_text(encoding="utf-8-sig"))
0021
0022
0023 def list_images(path: Path) -> List[Path]:
0024     if not path.exists():
0025         return []
0026     return sorted(
0027         item
0028         for item in path.iterdir()
0029         if item.is_file() and item.suffix.lower() in IMAGE_EXTS
0030     )
0031
0032
0033 def run_command(cmd: List[str]) -> None:
0034     print("[RUN]", " ".join(cmd))
0035     completed = subprocess.run(cmd, check=False)
0036     if completed.returncode != 0:
0037         raise RuntimeError(f"Command failed with code {completed.returncode}: {' '.join(cmd)}")
0038
0039
0040 def check_source_dirs(root_dir: Path) -> Tuple[bool, Dict[str, int]]:
0041     counts: Dict[str, int] = {}
0042     missing: List[Path] = []
0043
0044     for sample_type in SAMPLE_TYPES:
0045         target = root_dir / sample_type
0046         if not target.exists():
0047             missing.append(target)
0048             counts[sample_type] = 0
0049             continue
0050         counts[sample_type] = len(list_images(target))
0051         if counts[sample_type] == 0:
0052             print(f"[WARNING] {sample_type} 目录为空，相关分组指标可能无法稳定评估。")
0053
0054     if missing:
0055         print("[ERROR] v3_real 四类样本目录尚未准备完整: ")
0056         for item in missing:
0057             print(f" - {item}")
0058         print("请先运行: ")
0059         print(f"..\venv\Scripts\python.exe prepare_cache_v3_dirs.py")
0060         return False, counts
0061
0062     total = sum(counts.values())
0063     print("[INFO] cache_test_v3_real 样本统计: ")
0064     for sample_type in SAMPLE_TYPES:
0065         print(f" {sample_type}: {counts[sample_type]}")
0066     print(f" total: {total}")
0067
0068     if total == 0:
0069         print("[WARNING] 当前还没有 v3_real 测试图片，请先向四类目录补充图片。")
0070     elif total < 50:
0071         print("[WARNING] 当前样本数少于建议目标约 50 条，可先流程验证，正式汇报建议继续扩充。")
0072
0073     return True, counts
0074
0075
0076 def build_dataset_if_needed(
0077     python_exe: str,
0078     source_root: Path,
0079     dataset_dir: Path,
0080     rebuild_labels: bool,
0081 ) -> None:
0082     labels_path = dataset_dir / "labels.json"
0083     if labels_path.exists() and not rebuild_labels:
0084         print(f"[INFO] 已存在 labels.json，默认不覆盖: {labels_path}")
0085         return
0086
0087     run_command(
0088         [
0089             python_exe,
0090             "cache_dataset_auto_make.py",
0091             "--positive-dir",
0092             str(source_root / "positive"),
0093             "--near-positive-dir",
0094             str(source_root / "near_positive"),
0095             "--hard-negative-dir",
0096             str(source_root / "hard_negative"),
0097             "--negative-dir",
0098             str(source_root / "negative"),
0099             "--output-dir",
0100             str(dataset_dir),
0101             "--auto-fill-labels",
0102         ]
0103     )
0104
0105
0106 def inspect_labels(labels_path: Path) -> Dict[str, Any]:

```

```

0107     if not labels_path.exists():
0108         raise FileNotFoundError(f"labels.json 不存在: {labels_path}")
0109
0110     data = read_json(labels_path)
0111     samples = data.get("samples", [])
0112     counts = {sample_type: 0 for sample_type in SAMPLE_TYPES}
0113     empty_category = 0
0114     empty_query_text = 0
0115
0116     for sample in samples:
0117         sample_type = str(sample.get("sample_type") or "")
0118         if sample_type in counts:
0119             counts[sample_type] += 1
0120         if not str(sample.get("category") or "").strip():
0121             empty_category += 1
0122         if not str(sample.get("query_text") or "").strip():
0123             empty_query_text += 1
0124
0125     stats = {
0126         "total_samples": len(samples),
0127         "positive_count": counts["positive"],
0128         "near_positive_count": counts["near_positive"],
0129         "hard_negative_count": counts["hard_negative"],
0130         "negative_count": counts["negative"],
0131         "empty_category_count": empty_category,
0132         "empty_query_text_count": empty_query_text,
0133     }
0134
0135     print("[INFO] labels.json 样本统计: ")
0136     for key, value in stats.items():
0137         print(f"    {key}: {value}")
0138
0139     if empty_category or empty_query_text:
0140         print("[WARNING] 存在空 category 或 query_text, 请人工检查 labels.json 后再作为正式结果使用。")
0141
0142     return stats
0143
0144
0145 def run_label_check(python_exe: str, dataset_dir: Path) -> None:
0146     run_command(
0147         [
0148             python_exe,
0149             "check_cache_labels.py",
0150             "--labels",
0151             str(dataset_dir / "labels.json"),
0152             "--dataset-dir",
0153             str(dataset_dir),
0154             "--output",
0155             str(dataset_dir / "labels_check_report.md"),
0156         ]
0157     )
0158
0159
0160 def run_experiment(python_exe: str, dataset_dir: Path, output_dir: Path, thresholds: str) -> None:
0161     run_command(
0162         [
0163             python_exe,
0164             "experiment_cache_similarity.py",
0165             "--dataset-dir",
0166             str(dataset_dir),
0167             "--output-dir",
0168             str(output_dir),
0169             "--thresholds",
0170             thresholds,
0171         ]
0172     )
0173
0174
0175 def print_v3_todo() -> None:
0176     print("")
0177     print("v3_real 跑完后建议依次重新运行: ")
0178     print("1. 融合权重消融")
0179     print("2. 加权双阈值分析")
0180     print("3. 延迟对比分析")
0181     print("4. review 敏感性分析")
0182     print("5. 最终报告整合")
0183
0184
0185 def main() -> None:
0186     parser = argparse.ArgumentParser(description="One-click v3_real cache similarity experiment runner.")
0187     parser.add_argument("--source-root", default=str(DEFAULT_SOURCE_ROOT))
0188     parser.add_argument("--dataset-dir", default=str(DEFAULT_DATASET_DIR))
0189     parser.add_argument("--output-dir", default=str(DEFAULT_OUTPUT_DIR))
0190     parser.add_argument("--thresholds", default=DEFAULT_THRESHOLDS)
0191     parser.add_argument("--rebuild-labels", action="store_true")
0192     parser.add_argument("--python-exe", default=sys.executable)
0193     args = parser.parse_args()
0194
0195     source_root = Path(args.source_root)
0196     dataset_dir = Path(args.dataset_dir)

```

```

0197     output_dir = Path(args.output_dir)
0198
0199     ok, counts = check_source_dirs(source_root)
0200     if not ok:
0201         sys.exit(1)
0202
0203     if sum(counts.values()) == 0 and not (dataset_dir / "labels.json").exists():
0204         print("[INFO] 未发现图片且 labels.json 不存在, 暂不生成数据集。")
0205         print(f"v3_real 样本目录路径: {source_root}")
0206         print(f"v3_real 数据集输出路径: {dataset_dir}")
0207         print(f"v3_real 实验输出路径: {output_dir}")
0208         print("建议下一步: 用户向四类目录补充图片, 然后运行: ")
0209         print(".\\.venv\\Scripts\\python.exe run_cache_v3_experiment.py --rebuild-labels")
0210         sys.exit(0)
0211
0212     try:
0213         build_dataset_if_needed(
0214             python_exe=args.python_exe,
0215             source_root=source_root,
0216             dataset_dir=dataset_dir,
0217             rebuild_labels=args.rebuild_labels,
0218         )
0219         inspect_labels(dataset_dir / "labels.json")
0220         run_label_check(args.python_exe, dataset_dir)
0221         run_experiment(args.python_exe, dataset_dir, output_dir, args.thresholds)
0222         summary = read_json(output_dir / "summary.json")
0223     except Exception as exc:
0224         print(f"[ERROR] v3_real 实验运行失败: {exc}")
0225         sys.exit(1)
0226
0227     print("=" * 72)
0228     print("v3_real cache similarity experiment finished")
0229     print(f"v3_real 样本目录路径: {source_root}")
0230     print(f"v3_real 数据集输出路径: {dataset_dir}")
0231     print(f"v3_real 实验输出路径: {output_dir}")
0232     print(f"summary.csv: {output_dir / 'summary.csv'}")
0233     print(f"summary.json: {output_dir / 'summary.json'}")
0234     print(f"threshold_summary.csv: {output_dir / 'threshold_summary.csv'}")
0235     print(f"recommended_threshold: {summary.get('recommended_threshold')}")
0236     print_v3_todo()
0237     print("=" * 72)
0238
0239
0240 if __name__ == "__main__":
0241     main()

```